

# netsequence: Intelligent Change Sequencing for Network Topologies

Risk Analysis and Execution Planning for Network Mutations

---

Simon Knight, Ph.D.

March 2026 • Version 1.0

Last updated: March 18, 2026

**Abstract.** Network topology changes carry inherent risk: a poorly ordered migration can create transient black-holes, trigger cascading failures, or induce persistent routing oscillations. We present **NETSEQUENCE**, an orchestration layer that sits between structural diff generation (**NETCFG**) and operational deployment. Given a set of proposed graph mutations (a *MutationPlan*), **NETSEQUENCE** evaluates risk across six independent domains—single points of failure, traffic overload, BGP oscillation, IGP micro-loops, convergence time, and blast radius—each grounded in established graph-theoretic and network-analytic foundations (Tarjan’s biconnected components, Edmonds-Karp max-flow, Griffin-Shepherd-Wilfong dispute wheels, Motter-Lai cascades, and Fiedler’s algebraic connectivity). It then decomposes the plan into atomic change groups, constructs a dependency DAG, and produces a risk-weighted execution sequence with rollback plans and intermediate verification gates. We formalise the *Change Sequencing Problem*, prove it NP-hard via reduction from weighted job scheduling, and show that **NETSEQUENCE**’s greedy algorithm provides a monotonic-risk-reduction guarantee with bounded rollback at every intermediate state. Nine worked scenarios—including router replacement, IGP metric migration, BGP policy changes, and EVPN/VXLAN fabric migration—demonstrate that intelligent sequencing eliminates black-holes, micro-loops, and cascade exposure that naïve execution would cause. **NETSEQUENCE** is an orchestrator: it delegates graph algorithms to **NETASSURE**, simulation to **NETSIM** and **NETFLOWSIM**, and topology management to the Network Topology Engine (NTE).

## Contents

|           |                                                   |           |
|-----------|---------------------------------------------------|-----------|
| <b>1</b>  | <b>Introduction</b>                               | <b>2</b>  |
| <b>2</b>  | <b>System Architecture</b>                        | <b>3</b>  |
| <b>3</b>  | <b>Change Model</b>                               | <b>4</b>  |
| 3.1       | Multi-Layer Topology Representation . . . . .     | 4         |
| 3.2       | MVCC Shadow Topologies . . . . .                  | 4         |
| <b>4</b>  | <b>Problem Formulation</b>                        | <b>6</b>  |
| <b>5</b>  | <b>Risk Analysis</b>                              | <b>6</b>  |
| 5.1       | Domain 1: Single Point of Failure. . . . .        | 6         |
| 5.2       | Domain 2: Traffic Overload . . . . .              | 7         |
| 5.3       | Domain 3: BGP Oscillation. . . . .                | 8         |
| 5.4       | Domain 4: IGP Oscillation . . . . .               | 10        |
| 5.5       | Domain 5: Convergence Risk . . . . .              | 11        |
| 5.6       | Domain 6: Blast Radius. . . . .                   | 12        |
| 5.7       | Cross-Domain Risk Interactions. . . . .           | 13        |
| 5.8       | Risk Aggregation. . . . .                         | 13        |
| <b>6</b>  | <b>Change Sequencing</b>                          | <b>14</b> |
| 6.1       | Group Construction . . . . .                      | 14        |
| 6.2       | Make-Before-Break. . . . .                        | 15        |
| 6.3       | Dependency DAG . . . . .                          | 15        |
| 6.4       | Risk-Weighted Topological Sort . . . . .          | 15        |
| 6.5       | Execution with Verification Gates . . . . .       | 16        |
| <b>7</b>  | <b>Safe Network Transitions</b>                   | <b>16</b> |
| 7.1       | Update Paradigms . . . . .                        | 16        |
| 7.2       | Consistent Network Updates . . . . .              | 16        |
| 7.3       | Congestion-Free Updates . . . . .                 | 17        |
| 7.4       | Lossless IGP Migrations . . . . .                 | 17        |
| 7.5       | Dependency-Driven Scheduling. . . . .             | 17        |
| 7.6       | Network Synthesis . . . . .                       | 17        |
| <b>8</b>  | <b>Integration</b>                                | <b>18</b> |
| <b>9</b>  | <b>Report Schema</b>                              | <b>19</b> |
| <b>10</b> | <b>Extensibility</b>                              | <b>19</b> |
| <b>11</b> | <b>Related Work</b>                               | <b>20</b> |
| 11.1      | Network Verification. . . . .                     | 20        |
| 11.2      | BGP Stability Theory . . . . .                    | 20        |
| 11.3      | Network Resilience and Cascade Failures . . . . . | 20        |
| 11.4      | IGP Convergence and Fast Reroute . . . . .        | 20        |
| 11.5      | Safe Network Updates. . . . .                     | 21        |
| 11.6      | Network Change Management. . . . .                | 21        |
| 11.7      | Network Digital Twins . . . . .                   | 21        |
| <b>12</b> | <b>Naive vs. Intelligent Execution</b>            | <b>21</b> |
| 12.1      | Scenario: Backbone Maintenance Window . . . . .   | 21        |
| 12.2      | Naïve Execution . . . . .                         | 22        |
| 12.3      | Intelligent Execution. . . . .                    | 22        |
| 12.4      | Step-by-Step State Sequence. . . . .              | 22        |
| 12.5      | Comparison . . . . .                              | 22        |
| 12.6      | Risk Profile Visualisation . . . . .              | 22        |
| <b>13</b> | <b>Scenario: Data Centre Fabric Expansion</b>     | <b>25</b> |
| 13.1      | Baseline Topology. . . . .                        | 25        |
| 13.2      | Change Plan . . . . .                             | 25        |
| 13.3      | Risk Analysis. . . . .                            | 25        |
| <b>14</b> | <b>Scenario: IGP Metric Migration</b>             | <b>25</b> |
| 14.1      | Problem . . . . .                                 | 25        |

|           |                                                                                   |           |
|-----------|-----------------------------------------------------------------------------------|-----------|
| 14.2      | Naïve Approach. . . . .                                                           | 25        |
| 14.3      | Intelligent Approach. . . . .                                                     | 26        |
| <b>15</b> | <b>Scenario: BGP Peering Policy Change</b>                                        | <b>26</b> |
| 15.1      | Problem . . . . .                                                                 | 26        |
| 15.2      | Risk Detection . . . . .                                                          | 26        |
| 15.3      | Mitigation . . . . .                                                              | 26        |
| <b>16</b> | <b>Scenario: Dual-Homed Customer Migration</b>                                    | <b>26</b> |
| 16.1      | Topology . . . . .                                                                | 26        |
| 16.2      | Sequencing Challenge. . . . .                                                     | 26        |
| <b>17</b> | <b>Scenario: Ring Network Link Upgrade</b>                                        | <b>27</b> |
| 17.1      | Topology . . . . .                                                                | 27        |
| 17.2      | Risk Analysis . . . . .                                                           | 27        |
| 17.3      | Intelligent Approach . . . . .                                                    | 28        |
| 17.4      | Quantitative Comparison . . . . .                                                 | 28        |
| <b>18</b> | <b>Scenario: MPLS TE Tunnel Re-Optimisation</b>                                   | <b>28</b> |
| 18.1      | Problem . . . . .                                                                 | 28        |
| 18.2      | Risk Analysis . . . . .                                                           | 28        |
| <b>19</b> | <b>Scenario: Graceful BGP Shutdown</b>                                            | <b>29</b> |
| 19.1      | Problem . . . . .                                                                 | 29        |
| 19.2      | Topology . . . . .                                                                | 29        |
| 19.3      | Naïve Approach. . . . .                                                           | 29        |
| 19.4      | Intelligent Approach . . . . .                                                    | 29        |
| <b>20</b> | <b>Scenario: EVPN/VXLAN Fabric Migration</b>                                      | <b>30</b> |
| 20.1      | Problem . . . . .                                                                 | 30        |
| 20.2      | Risk Analysis . . . . .                                                           | 30        |
| 20.3      | Execution Sequence . . . . .                                                      | 30        |
| <b>21</b> | <b>Formal Safety Properties</b>                                                   | <b>30</b> |
| 21.1      | Complexity Analysis . . . . .                                                     | 31        |
| <b>22</b> | <b>Execution Engine State Machine</b>                                             | <b>32</b> |
| <b>23</b> | <b>Sensitivity Analysis</b>                                                       | <b>32</b> |
| 23.1      | Threshold Sensitivity . . . . .                                                   | 32        |
| 23.2      | Weight Vector Impact . . . . .                                                    | 32        |
| 23.3      | Network Topology Characteristics. . . . .                                         | 32        |
| <b>24</b> | <b>Comparison with Existing Approaches</b>                                        | <b>34</b> |
| <b>25</b> | <b>Lessons from Production Failures</b>                                           | <b>34</b> |
| <b>26</b> | <b>End-to-End Worked Example</b>                                                  | <b>35</b> |
| 26.1      | Input . . . . .                                                                   | 35        |
| 26.2      | Group Construction . . . . .                                                      | 35        |
| 26.3      | Shadow Topology Chain. . . . .                                                    | 35        |
| 26.4      | Risk Analysis Results . . . . .                                                   | 35        |
| 26.5      | Dependency DAG and Execution. . . . .                                             | 36        |
| 26.6      | Report Output . . . . .                                                           | 36        |
| <b>27</b> | <b>Limitations and Future Work</b>                                                | <b>36</b> |
| <b>28</b> | <b>Future Directions: Towards Autonomous Orchestration</b>                        | <b>36</b> |
| 28.1      | Automated State Synthesis (Temporary Bridging). . . . .                           | 37        |
| 28.2      | Live Telemetry-Driven Verification Gates . . . . .                                | 37        |
| 28.3      | Continuous (Temporal) Risk Modeling. . . . .                                      | 37        |
| 28.4      | Synthesized Canary Deployments (Blast-Radius Containment) . . . . .               | 37        |
| 28.5      | Optimal Solver Fallback (ILP) . . . . .                                           | 38        |
| 28.6      | Explainable AI (xAI) for Risk Remediation . . . . .                               | 38        |
| 28.7      | Chaos Engineering on Intermediate States ( $N - 1$ Migration Resilience). . . . . | 38        |
| 28.8      | Robust Sequencing under Traffic Uncertainty . . . . .                             | 38        |
| 28.9      | Semantic Reconciliation (Dynamic Roll-Forward / Safe Harbors). . . . .            | 39        |
| 28.10     | Federated Multi-Domain Orchestration (Privacy-Preserving). . . . .                | 39        |

|           |                                                                         |           |
|-----------|-------------------------------------------------------------------------|-----------|
| 28.11     | Out-of-Band Drift Detection and Dynamic Re-Planning. . . . .            | 40        |
| 28.12     | Energy-Aware and Carbon-Optimized Sequencing . . . . .                  | 40        |
| 28.13     | Multi-Objective Optimization (Risk vs. Time) . . . . .                  | 40        |
| 28.14     | Continuous Topology Defragmentation (Zero-Window Refactoring). . . . .  | 40        |
| 28.15     | Post-Quantum Cryptography (PQC) Transition Sequencing. . . . .          | 41        |
| 28.16     | Adversarial Risk Surface Modeling (Security-Aware Sequencing) . . . . . | 41        |
| 28.17     | Algorithmic Governance (Policy-as-Code Integration) . . . . .           | 42        |
| <b>29</b> | <b>Conclusion</b>                                                       | <b>42</b> |
| <b>A</b>  | <b>Algorithms</b>                                                       | <b>46</b> |

## Contents

---

## 1 Introduction

Operational networks evolve continuously: links are provisioned and decommissioned, routing policies are adjusted, and capacity is redistributed. Each change carries risk. Studies of large-scale networks have shown that planned maintenance activities account for a significant fraction of network incidents [1–4], and that the *ordering* of changes—not just their content—materially affects the probability of service disruption.

Consider a seemingly simple operation: decommissioning a transit router and migrating its traffic to a newly provisioned replacement. This involves at least six distinct mutations (add node, add links, shift routing metrics, drain traffic, remove old links, remove old node) with complex interdependencies. If the old links are removed before the new ones carry traffic, a transient black-hole appears. If routing metrics are changed before the new path is fully converged, micro-loops form. If the new node creates an articulation point in the intermediate topology, a single additional failure could partition the network. These risks are invisible to tools that only compare the initial and final configurations.

Existing tooling addresses *what* to change (configuration management, structural diffing [5]) and *whether* the result is correct (formal verification [6–8], invariant checking). Safe update techniques [9–11] address specific properties (per-packet consistency, congestion freedom) but do not provide unified multi-domain risk assessment. The missing layer is *planning*: given a set of proposed mutations, what is the risk profile across *all* relevant domains, and in what order should changes be applied to minimise disruption?

NETSEQUENCE fills this gap. It operates on the *MutationPlan*—a sequence of graph-level operations produced by NETCFG’s DiffEngine—and produces a structured risk report with an ordered execution sequence. Its key insight is that *intermediate states matter*: risks that exist only during the transition but disappear in the final topology are often the most dangerous, precisely because they are hardest to anticipate.

### Contributions.

1. A formal change model based on NTE graph mutations with MVCC shadow topologies for intermediate-state analysis (section 3).
2. A formal problem definition (the Change Sequencing Problem) with an NP-hardness proof and greedy approximation analysis (section 4).
3. Six independent risk analyzers with concrete mathematical foundations, covering structural, protocol, and traffic domains (section 5).
4. A change decomposition and sequencing algorithm that respects layer coupling, protocol dependencies, and make-before-break constraints (section 6).
5. Safety guarantees: a main theorem establishing per-transition safety, greedy optimality, and bounded rollback (section 21).
6. A survey of safe network update techniques and how NETSEQUENCE extends them to multi-domain risk assessment (section 7).
7. An extensible analyzer trait model with planned WASM plugin support (section 10).

### Notation

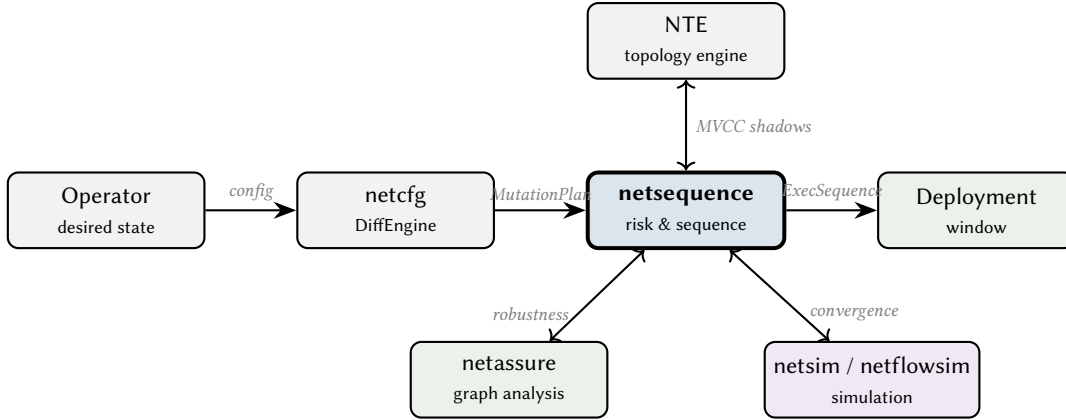
Table 1 summarises the principal symbols used throughout the paper.

**Table 1.** Summary of notation.

| Symbol               | Description                                          |
|----------------------|------------------------------------------------------|
| $G = (V, E)$         | Network topology graph                               |
| $G_0$                | Baseline (pre-change) topology                       |
| $G^*$                | Target (post-change) topology                        |
| $G_k$                | Intermediate topology after applying $g_k$           |
| $\delta_i$           | Atomic mutation (AddNode, RemoveNode, etc.)          |
| $\mathcal{M}$        | MutationPlan = $(\delta_1, \dots, \delta_m)$         |
| $g_k$                | Change group (set of coupled mutations)              |
| $g_k^{-1}$           | Inverse (rollback) of group $g_k$                    |
| $D = (G, E)$         | Dependency DAG over change groups                    |
| $r(g)$               | Aggregate risk score for group $g$                   |
| $r_d(G_k, g)$        | Risk score from domain $d$ for transition $(G_k, g)$ |
| $w_d$                | Weight for risk domain $d$ ; $\sum_d w_d = 1$        |
| $\theta$             | Utilisation alert threshold (default 0.8)            |
| $c(e), d(e)$         | Capacity and demand on edge $e$                      |
| $u(e)$               | Link utilisation = $d(e)/c(e)$                       |
| $\lambda_2(G)$       | Fiedler value (algebraic connectivity) of $G$        |
| $\kappa(G)$          | Vertex connectivity of $G$                           |
| $f_{\max}(s, t)$     | Maximum flow from $s$ to $t$                         |
| $T_{\text{conv}}(g)$ | Convergence time bound for group $g$                 |

## 2 System Architecture

Figure 1 shows NETSEQUENCE’s position in the network automation ecosystem.



**Figure 1.** Ecosystem positioning of NETSEQUENCE. The tool receives a MutationPlan from NETCFG, queries analysis and simulation subsystems, and emits an ordered execution sequence.

NETSEQUENCE is an *orchestrator*: it contains no graph algorithms of its own. Structural analysis is delegated to NETASSURE (robustness metrics, spectral analysis, cascade simulation), packet-level simulation to NETSIM (protocol convergence), flow-level simulation to NETFLOWSIM (traffic redistribution), and topology state management to the NTE.

Figure 2 shows the internal processing pipeline.

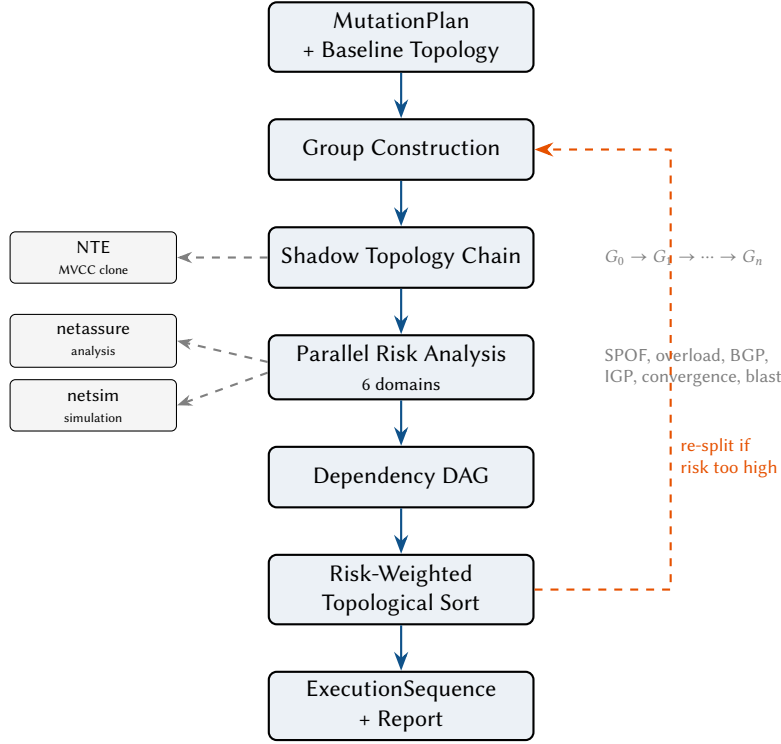


Figure 2. Internal processing pipeline of NETSEQUENCE.

### 3 Change Model

**Definition 3.1 (MutationPlan).** A MutationPlan  $\mathcal{M} = (\delta_1, \delta_2, \dots, \delta_m)$  is an ordered sequence of graph mutations  $\delta_i \in \{\text{ADDNODE}, \text{REMOVENODE}, \text{ADDEDGE}, \text{REMOVEEDGE}\}$  that transforms a baseline topology  $G_0$  into a desired topology  $G^*$ .

Each mutation operates on the NTE graph—a directed `petgraph::StableGraph` with typed nodes (carrying optional layer and attribute metadata) and typed edges (carrying optional `edge_type` for cross-layer dependencies).

**Mutation types.** The four operations have precise graph-theoretic semantics:

- `ADDNODE( $n, \ell, \mathbf{a}$ )`: insert node  $n$  at layer  $\ell$  with attributes  $\mathbf{a}$ .
- `REMOVENODE( $n$ )`: delete node  $n$  and all incident edges (both intra-layer and cross-layer dependency edges).
- `ADDEDGE( $u, v, t, \mathbf{a}$ )`: add edge from  $u$  to  $v$  with type  $t \in \{\text{intra}, \text{dependency}\}$ .
- `REMOVEEDGE( $e$ )`: delete edge  $e$  by identifier.

Figure 3 shows a concrete MutationPlan for migrating traffic from an old path (R1–R3–R4) to a new path (R1–R5–R4) while decommissioning R3.

#### 3.1 Multi-Layer Topology Representation

The NTE graph encodes multiple network abstraction layers in a single directed graph—a standard approach in multi-layer network modelling [12, 13]. Cross-layer dependencies are represented by edges with `edge_type` = "dependency", forming the hierarchy shown in fig. 4.

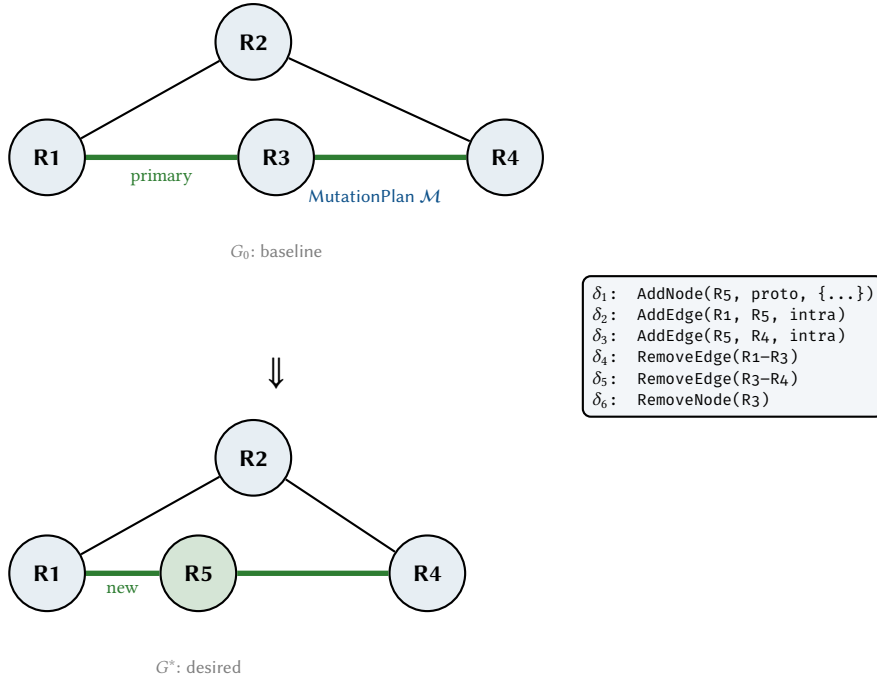
Figure 5 shows a concrete multi-layer graph fragment. Removing the physical port `ge-0/0/1` cascades upward: the logical interface and its OSPF adjacency must be removed first, along with any services depending on that adjacency. This cascade is automatically captured by the dependency edges in the NTE graph.

#### 3.2 MVCC Shadow Topologies

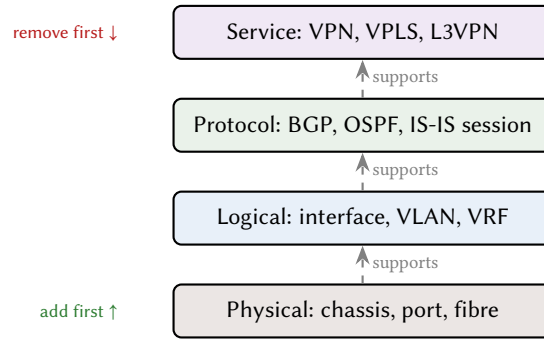
Borrowing from database multi-version concurrency control (MVCC) [14, 15], NETSEQUENCE constructs a chain of *shadow topologies* representing every intermediate state during execution, rather than analysing only the baseline  $G_0$  and final state  $G^*$ :

$$G_0 \xrightarrow{g_1} G_1 \xrightarrow{g_2} G_2 \cdots \xrightarrow{g_n} G_n = G^*$$

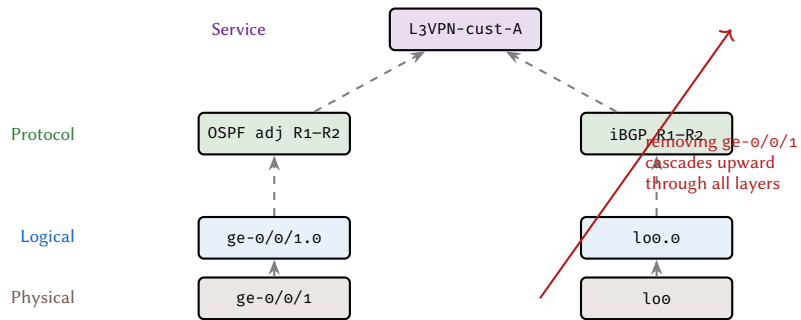
where each  $g_k$  is a *change group* (an atomic subset of  $\mathcal{M}$ ). This approach catches transient risks that exist only during migration but disappear in the final state.



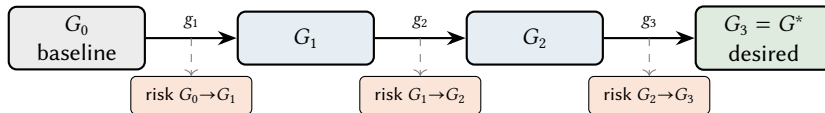
**Figure 3.** A concrete MutationPlan: migrate traffic from R1–R3–R4 to R1–R5–R4 and decommission R3. The six mutations must be ordered to maintain connectivity (make-before-break:  $\delta_1$ – $\delta_3$  before  $\delta_4$ – $\delta_6$ ).



**Figure 4.** Multi-layer topology hierarchy. Additions proceed bottom-up; removals proceed top-down.

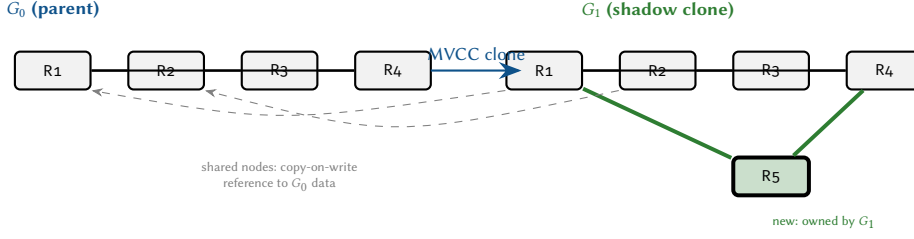


**Figure 5.** Multi-layer graph fragment showing cross-layer dependencies. Removing physical port ge-0/0/1 requires first removing the OSPF adjacency and L3VPN service above it.



**Figure 6.** Shadow topology chain with per-transition risk analysis.

Figure 7 shows the MVCC clone operation in detail. Each shadow topology shares the majority of its structure with the parent; only mutated nodes and edges are copied, making the clone operation  $O(|g_k|)$  rather than  $O(|V| + |E|)$  in practice.



**Figure 7.** MVCC shadow clone operation.  $G_1$  shares unchanged nodes (R1–R4) with  $G_0$  via copy-on-write references; only new or mutated elements (R5, new edges) are exclusively owned by the shadow. This makes clone creation proportional to the change group size.

## 4 Problem Formulation

We now formalise the optimisation problem that `NETSEQUENCE` solves.

**Definition 4.1 (Change Sequencing Problem).** *Given a baseline topology  $G_0 = (V_0, E_0)$ , a MutationPlan  $\mathcal{M}$  decomposed into change groups  $\{g_1, \dots, g_n\}$ , a dependency DAG  $D = (G, E_D)$  over these groups, a risk function  $r : 2^{V \cup E} \times 2^{V \cup E} \rightarrow \mathbb{R}_{\geq 0}$  mapping topology transitions to risk scores, and a safety threshold  $\tau \in \mathbb{R}_{>0}$ , find a permutation  $\pi : [n] \rightarrow [n]$  such that:*

1. **Dependency respect:** if  $(g_i, g_j) \in E_D$  then  $\pi^{-1}(i) < \pi^{-1}(j)$ .
2. **Per-transition safety:** for every consecutive pair,  $r(G_{k-1}, G_k) \leq \tau$ .
3. **Minimum cumulative exposure:** the objective  $\sum_{k=1}^n r(G_{k-1}, G_k)$  is minimised.

**Lemma 4.1 (NP-hardness).** *The Change Sequencing Problem (definition 4.1) is NP-hard in general.*

*Proof.* We reduce from the NP-hard weighted job scheduling problem on precedence-constrained DAGs [16, 17]. Given an instance of weighted job scheduling with precedence constraints  $(J, \prec, w)$ , map each job  $j_i$  to a change group  $g_i$  with risk  $r(G_{k-1}, G_k) = w(j_i)$ , and precedence  $j_i \prec j_j$  to dependency  $(g_i, g_j) \in E_D$ . The minimum-exposure permutation in the CSP instance corresponds exactly to a minimum weighted completion time schedule. Since the latter is NP-hard [17] (problem SS9), the CSP is NP-hard.  $\square$

**Greedy approximation.** `NETSEQUENCE`’s risk-weighted topological sort (algorithm 2) is a greedy heuristic using Kahn’s algorithm [18] with a min-priority queue keyed by risk score. For the related problem of minimising weighted completion time on a single machine with precedence constraints, the greedy priority-based approach achieves a 2-approximation [19, 20]. While the risk-minimisation objective differs from classic completion time, the structural similarity provides confidence that the greedy approach produces near-optimal results in practice: the algorithm always selects the lowest-risk ready group, avoiding the worst-case concentration of high-risk transitions early in the sequence.

## 5 Risk Analysis

`NETSEQUENCE` evaluates six independent risk domains (fig. 8). Each analyzer implements a common `RiskAnalyzer` trait and receives the baseline and proposed topologies. Analyzers run in parallel via `rayon`.

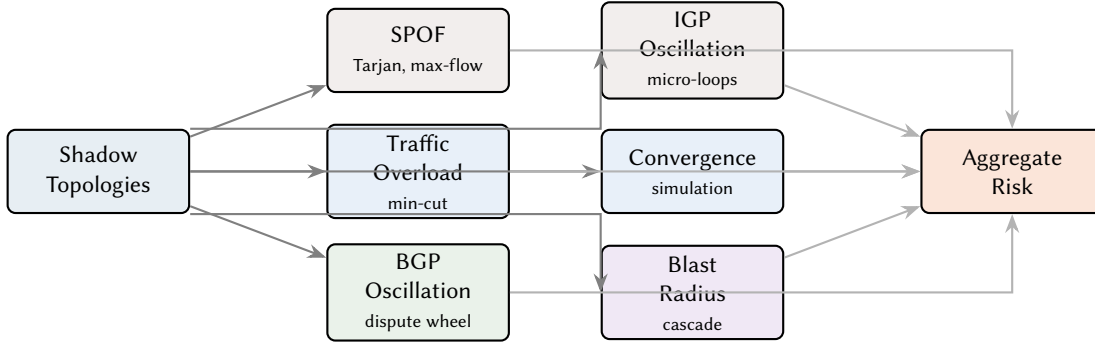
### 5.1 Domain 1: Single Point of Failure

**Articulation points.** A vertex  $v$  in a connected graph  $G = (V, E)$  is an *articulation point* if  $G \setminus \{v\}$  is disconnected. We compute articulation points using Tarjan’s algorithm [21, 22] in  $O(|V| + |E|)$  time, delegating to `netassure_analyze::robustness`.

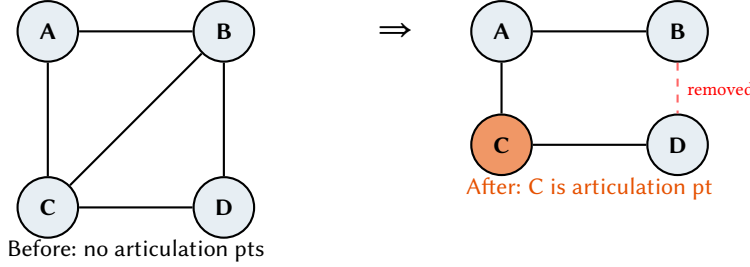
**Path diversity.** By Menger’s theorem [23], the maximum number of node-disjoint  $(s, t)$ -paths equals the minimum vertex cut separating  $s$  from  $t$ . For critical source–destination pairs  $(s, t)$ , we compute this via Edmonds-Karp max-flow [24–26] on the unit-capacity node-split graph:

$$\text{diversity}(s, t) = \text{max-flow}(G_{\text{unit}}, s, t) \quad (1)$$

Figure 9 illustrates how removing a link can create a new articulation point.



**Figure 8.** Six independent risk domains evaluated in parallel.



**Figure 9.** Removing edge B–D creates articulation point C (removing C disconnects {A,B} from {D}).

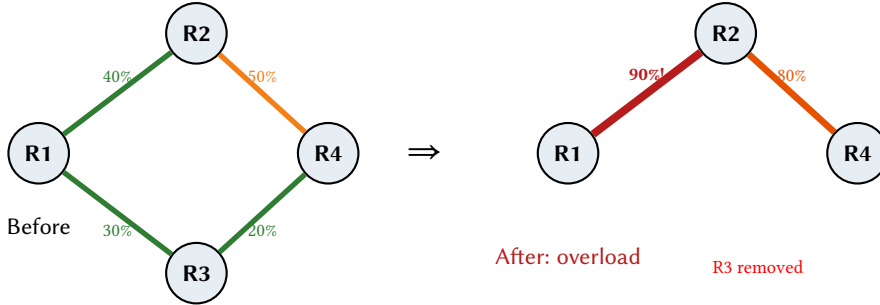
**Risk score.**

$$R_{\text{SPOF}} = \frac{|\Delta\text{AP}| + \Delta_{\text{bridges}}}{\max(|V|, 1)} + \sum_{(s,t) \in C} \max(0, \text{div}_0(s, t) - \text{div}^*(s, t)) \quad (2)$$

where  $\Delta\text{AP}$  is the set of new articulation points and  $C$  is the set of operator-designated critical pairs.

## 5.2 Domain 2: Traffic Overload

Construct a capacity graph where each edge  $e$  carries capacity  $c(e)$  and current demand  $d(e)$ . After applying mutations, traffic redistributes. We compute the utilisation  $u(e) = d_{\text{redistributed}}(e)/c(e)$  and flag edges exceeding threshold  $\theta$  (default 0.8).



**Figure 10.** Removing R3 forces traffic onto R1–R2, pushing utilisation from 40% to 90%—above the 80% threshold.

Traffic redistribution is simulated via `NETFLOWSIM`’s dynamic simulation engine with event-driven topology changes [25, 26]. Link capacities and TE attributes are derived from OSPF-TE extensions [27].

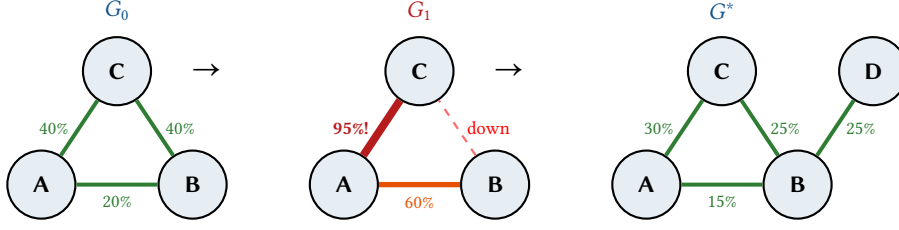
### 5.2.1 Transient vs. Steady-State Overload

A critical distinction is between *steady-state* overload (present in the final topology  $G^*$ ) and *transient* overload (present only during intermediate states  $G_k$ ). Transient overload is particularly dangerous because it may not be visible in before/after analysis.

Figure 11 shows a scenario where an intermediate state creates overload that exists in neither the baseline nor the target topology.

### 5.2.2 ECMP Redistribution

When multiple equal-cost paths exist, traffic distributes across them via ECMP hashing. Removing one ECMP next-hop causes an immediate  $1/n \rightarrow 1/(n-1)$  load increase on the remaining paths. For  $n = 2$  (dual-homed links), this is a 100% increase on the surviving path. `NETSEQUENCE` computes the ECMP group membership for each prefix in every intermediate topology and flags transitions where ECMP group reduction would push utilisation above threshold.



Transient overload: B-C down before D added.  
A-C carries 95% load in intermediate state only.

**Figure 11.** Transient overload scenario. The intermediate state  $G_1$  (B-C link removed before new node D is added) causes A-C overload that exists in neither  $G_0$  nor  $G^*$ . Re-ordering to add D first eliminates the transient risk.

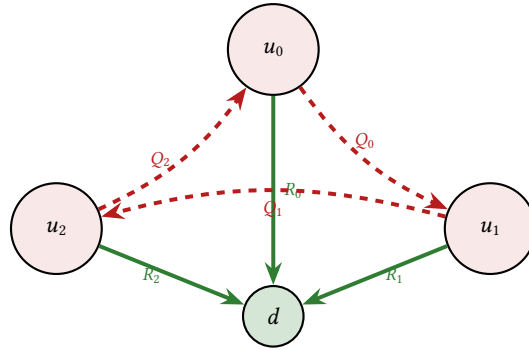
**Risk score.**

$$R_{\text{overload}} = \sum_{e \in E'} \max(0, u(e) - \theta) \cdot w(e) \quad (3)$$

### 5.3 Domain 3: BGP Oscillation

BGP convergence is not guaranteed under arbitrary routing policies. Griffin, Shepherd, and Wilfong [28] showed that BGP convergence depends on the absence of *dispute wheels* in the policy configuration. Gao and Rexford [29] later showed that commercially-motivated policies (customer/provider/peer relationships) guarantee convergence, but arbitrary policies may not. Sobrinho [30] provided an algebraic framework that unifies these results: convergence is guaranteed when the routing algebra is *strictly monotone*—i.e., extending a path never improves its preference.

**Definition 5.1 (Dispute Wheel [31]).** A dispute wheel of order  $k$  consists of nodes  $u_0, \dots, u_{k-1}$ , paths  $R_i$  (preferred by  $u_i$ ) and  $Q_i$  (through  $u_{i+1 \bmod k}$ ), where each  $u_i$  prefers  $R_i$  over  $Q_i$  but  $R_i$  is available only if  $u_{i-1 \bmod k}$  selects  $Q_{i-1}$ .



Dispute wheel: BGP will not converge

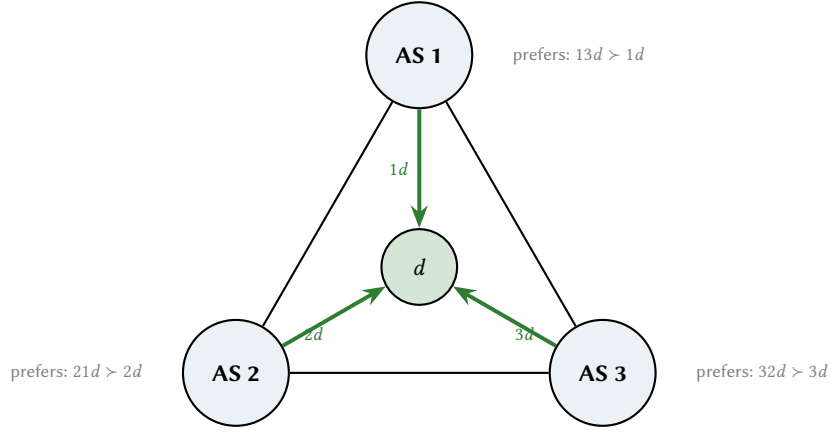
**Figure 12.** A dispute wheel of order 3. Each AS prefers its direct path  $R_i$  but that path depends on the predecessor's choice, creating a cyclic dependency.

#### 5.3.1 The Bad Gadget

The canonical example of BGP divergence is the *Bad Gadget* [31], a minimal topology where three ASes share conflicting preferences over routes to a common destination  $d$ . Figure 13 shows the topology and policy structure. Each AS  $i$  has a direct path to  $d$  and a two-hop path through its clockwise neighbour; each prefers the two-hop path. The resulting oscillation cycle is:

1. All three ASes initially select their direct paths ( $1d, 2d, 3d$ ).
2. AS 1 sees AS 3 using  $3d$  and switches to the preferred two-hop path  $13d$ . Similarly AS  $2 \rightarrow 21d$  and AS  $3 \rightarrow 32d$ .
3. But now AS 1's path through AS 3 (i.e.  $132d$ ) is longer than the direct path, so AS 1 reverts to  $1d$ . The cycle repeats.

The Bad Gadget is the simplest *DISAGREE* instance: Griffin and Wilfong [31] showed that even two ASes with conflicting preferences over a single destination can diverge, though the Bad Gadget is the standard pedagogical example.



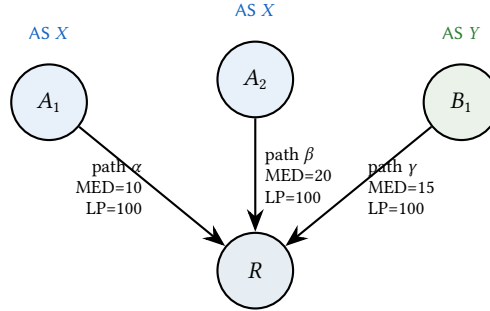
No stable assignment exists.  
BGP oscillates indefinitely.

**Figure 13.** The Bad Gadget [31]: three ASes with circular next-hop preferences. Each AS prefers the two-hop path through its clockwise neighbour over its direct path to  $d$ . No stable path assignment exists, and BGP oscillates indefinitely.

### 5.3.2 MED-Induced Oscillations

A subtler class of BGP oscillation arises from the Multi-Exit Discriminator (MED) attribute [32, 33]. Unlike local-preference and AS-path-length, MED is compared only across paths from the *same* neighbouring AS. This makes the BGP decision process non-deterministic: the set of candidate paths considered at each step depends on which paths have already been filtered, violating the independence of irrelevant alternatives.

Figure 14 illustrates a topology where MED comparison causes persistent oscillation. Router  $R$  receives three paths to prefix  $p$  from two upstream ASes. The MED comparison is only valid within each AS, but the interaction between the two comparison groups creates a cycle.



MED comparison:  $\alpha > \beta$  (same AS),  
but  $\beta$  vs  $\gamma$  depends on  $\alpha$ 's presence

**Figure 14.** MED-induced oscillation scenario. Paths  $\alpha$  and  $\beta$  are from AS X (MED-comparable); path  $\gamma$  is from AS Y. The best-path selection depends on which paths are currently in the RIB, creating a non-deterministic decision process.

### 5.3.3 Sobrinho's Routing Algebras

Sobrinho [30, 34] provides the most general sufficient condition for BGP convergence. A *routing algebra*  $(\Sigma, \oplus, \preceq, \phi, 0, \bar{0})$  consists of a set of *signatures*  $\Sigma$ , an extension operation  $\oplus$  (applying a policy), a preference order  $\preceq$ , and distinguished elements for the trivial path and unreachable.

**Definition 5.2 (Strict Monotonicity [30]).** A routing algebra is strictly monotone if for all labels  $\ell$  and all non-trivial signatures  $\sigma$ :  $\ell \oplus \sigma \prec \sigma$ . That is, extending a path by one hop always makes it strictly less preferred.

**Lemma 5.1 (Sobrinho [30]).** If the routing algebra is strictly monotone, then the path-vector protocol converges to a unique stable solution for every network topology.

Standard BGP with local-preference and shortest-AS-path is strictly monotone. Policies involving MED, route redistribution across IGP/BGP boundaries [35], or complex community-based filtering can violate monotonicity.

#### 5.3.4 Detection in netsequence

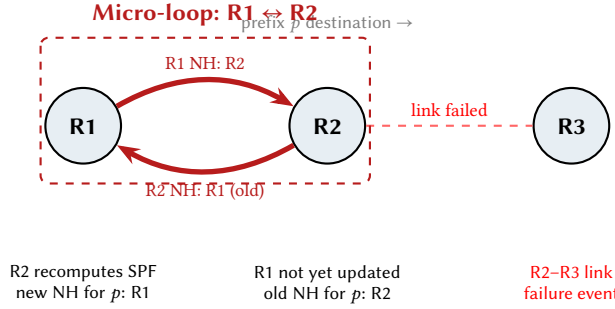
NETSEQUENCE’s BGP oscillation analyzer performs three checks:

1. **Dispute wheel search:** constructs the SPP graph from extracted policies and searches for dispute wheels [28].
2. **MED anomaly detection:** identifies routers receiving paths from multiple ASes where MED comparison is active, flagging topologies matching known oscillation patterns [32, 33].
3. **Monotonicity check:** for policy changes in the MutationPlan, verifies that the resulting routing algebra remains strictly monotone [30], checking that no policy violates the condition  $\ell \oplus \sigma \prec \sigma$ .

Detection is delegated to `netassure_verify`, which maintains an abstract policy graph extracted from the NTE topology. Feamster and Balakrishnan’s `rcc` [36] approach—static analysis of BGP configurations to detect faults—informs the design of policy extraction.

#### 5.4 Domain 4: IGP Oscillation

During IGP reconvergence, routers update their FIBs asynchronously, creating transient forwarding loops—*micro-loops* [37, 38]. François and Bonaventure [39] showed that ordered FIB updates can eliminate micro-loops; François’s thesis [40] provides the complete theoretical treatment. NETSEQUENCE assesses whether the proposed changes create conditions for micro-loops and estimates the convergence risk.



**Figure 15.** Micro-loop formation: R2 has recomputed its SPF (new next-hop for prefix  $p$  is R1) but R1 still uses R2 as next-hop (old FIB). Packets for  $p$  loop between R1 and R2 until R1 converges.

##### 5.4.1 Convergence Time Decomposition

IGP convergence after a topology change consists of three phases [40, 41]:

$$T_{\text{conv}} = T_{\text{detect}} + T_{\text{SPF}} + T_{\text{FIB}} \quad (4)$$

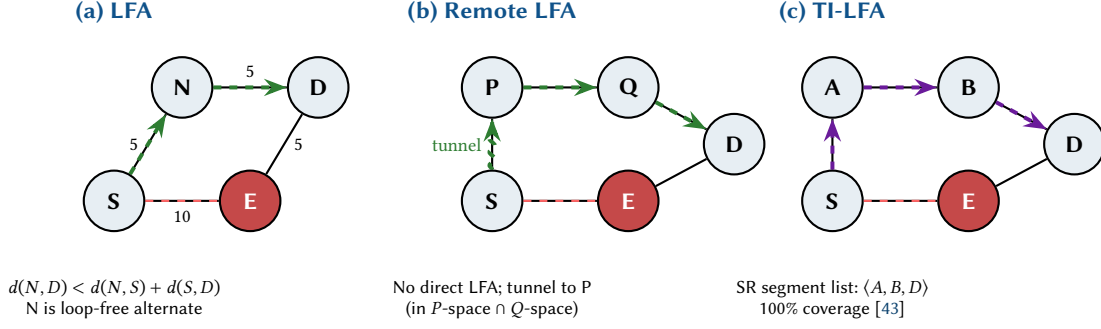
1. **Detection** ( $T_{\text{detect}}$ ): Time for the change to be detected, either via hello timer expiry (seconds) or carrier-loss (milliseconds). For planned changes in NETSEQUENCE, this is typically instantaneous since LSA/LSP generation is triggered by the mutation itself.
2. **SPF computation** ( $T_{\text{SPF}}$ ): Dijkstra’s algorithm runs in  $O(|V| \log |V| + |E|)$  time, but includes flooding delay proportional to network diameter:  $T_{\text{SPF}} = d \cdot t_{\text{hop}} + t_{\text{dijkstra}}$ .
3. **FIB installation** ( $T_{\text{FIB}}$ ): Each changed prefix requires a FIB update; total time is  $n_{\text{changed}} \cdot t_{\text{entry}}$ , where  $t_{\text{entry}}$  depends on the forwarding hardware (TCAM, hash table, etc.).

The micro-loop vulnerability window is the period where some routers have completed SPF but others have not. Its duration depends on the variance in  $T_{\text{SPF}}$  across routers in the network—higher variance means longer exposure.

##### 5.4.2 IP Fast Reroute Mechanisms

Modern networks deploy IP Fast Reroute (IPFRR) [42] to provide sub-50 ms protection against link and node failures. NETSEQUENCE evaluates IPFRR coverage to assess whether micro-loops will be mitigated by existing protection mechanisms. Figure 16 compares the three main approaches.

1. **Loop-Free Alternates (LFA)** [44]: for a source  $S$  with primary next-hop  $E$  and neighbour  $N$ ,  $N$  is an LFA if  $d(N, D) < d(N, S) + d(S, D)$ . Coverage depends on topology; typical networks achieve 70–95% prefix coverage [47].
2. **Remote LFA** [45]: extends coverage by tunnelling to a node in the intersection of  $P$ -space (nodes reachable from  $S$  without traversing  $S-E$ ) and  $Q$ -space (nodes from which  $D$  is reachable without traversing  $S-E$ ).



**Figure 16.** IP Fast Reroute mechanisms. (a) LFA [44]: direct loop-free alternate neighbour. (b) Remote LFA [45]: tunnel to a node in the intersection of  $P$ -space and  $Q$ -space. (c) TI-LFA [43, 46]: Segment Routing provides 100% topology-independent coverage via explicit segment lists.

3. **TI-LFA** [43]: uses Segment Routing [46] to compute explicit backup paths as segment lists, achieving 100% coverage independent of topology.

`NETSEQUENCE` computes LFA coverage for each intermediate topology state: if coverage drops below a configurable threshold (e.g., the network lacks TI-LFA deployment), the IGP risk score is elevated.

#### 5.4.3 Ordered FIB Updates

François and Bonaventure [39] proved that micro-loops can be completely avoided by computing a *safe FIB update ordering*. The key insight is that a router  $R$  can safely update its FIB for destination  $D$  if all routers on  $R$ 's new path to  $D$  have already updated (or their old path does not traverse  $R$ ). RFC 5715 [48] formalises this as the *loop-free convergence* framework.

For planned changes (as opposed to failures), `NETSEQUENCE` can pre-compute the safe ordering and embed it in the execution sequence, ensuring that each change group's IGP mutations are applied in an order that eliminates micro-loops entirely.

#### Risk score.

$$R_{\text{IGP}} = w_{\text{loop}} \cdot P_{\text{loop}} + w_{\text{cov}} \cdot (1 - C_{\text{LFA}}) + w_{\text{conv}} \cdot \frac{T_{\text{conv}}}{T_{\text{budget}}} \quad (5)$$

where  $P_{\text{loop}}$  is the estimated micro-loop probability (from `NETSIM` simulation or analytical model [37]),  $C_{\text{LFA}}$  is the IPFRR prefix coverage ratio, and  $T_{\text{budget}}$  is the operator-specified convergence time budget.

### 5.5 Domain 5: Convergence Risk

This domain uses `NETSIM`'s packet-level simulation to empirically measure convergence. The simulator runs the proposed changes as scheduled events and monitors convergence by checking `Device::fib_changed()` across all devices (each simulation tick = 10 ms). FIB churn hotspots—devices experiencing disproportionate route instability—are identified.

#### 5.5.1 Convergence Simulation Methodology

For each transition  $G_{k-1} \rightarrow G_k$ , `NETSIM` executes a three-phase simulation (fig. 17):

1. **Steady state:** run until all protocols have converged and FIBs are stable (baseline measurement).
2. **Event injection:** apply the mutations in change group  $g_k$  as timed events (link-state advertisements, BGP updates, interface state changes).
3. **Convergence monitoring:** track FIB changes per device per tick until stability is re-established or a timeout (default: 60 s simulation time) is reached.

#### 5.5.2 FIB Churn Hotspots

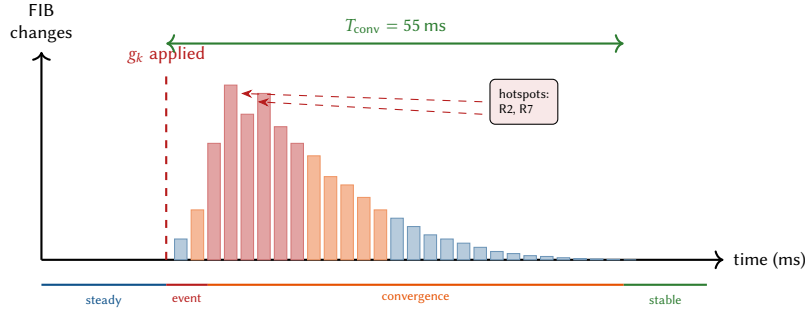
A device  $v$  is a *FIB churn hotspot* if its per-device churn exceeds a threshold relative to the network-wide average:

$$\text{hotspot}(v) = \frac{\text{churn}(v)}{\frac{1}{|V|} \sum_{u \in V} \text{churn}(u)} > \theta_{\text{churn}} \quad (6)$$

where  $\text{churn}(v) = \sum_t |\Delta \text{FIB}_v(t)|$  counts the total FIB entries modified across all ticks. Hotspots indicate devices at risk of control-plane CPU exhaustion or forwarding-plane instability during the transition.

#### Risk score.

$$R_{\text{conv}} = w_1 \cdot \frac{T_{\text{conv}}}{T_{\text{budget}}} + w_2 \cdot \frac{|\text{hotspots}|}{|V|} + w_3 \cdot [T_{\text{conv}} > T_{\text{timeout}}] \quad (7)$$

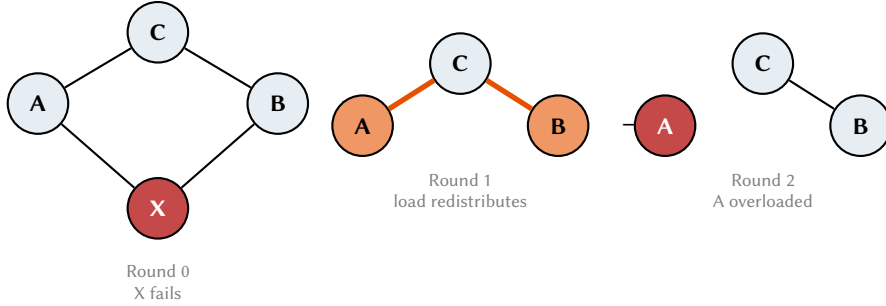


**Figure 17.** Convergence timeline after applying change group  $g_k$ . FIB churn peaks immediately after the event and decays exponentially. Devices R2 and R7 are identified as hotspots due to disproportionate churn. Convergence time  $T_{\text{conv}} = 55$  ms in this example.

The third term is a binary penalty if convergence was not achieved within the simulation timeout, indicating potential divergence.

### 5.6 Domain 6: Blast Radius

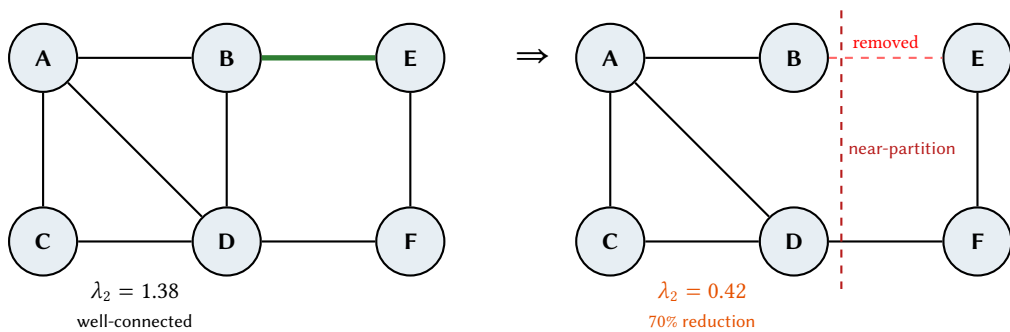
**Cascade simulation.** The Motter-Lai model [49] simulates load redistribution after failures. When a component fails, its load (proportional to betweenness centrality [50]) redistributes to neighbours; if a neighbour exceeds its tolerance, it fails too, potentially triggering a cascade (fig. 18).



**Figure 18.** Motter-Lai cascade propagation. X fails (round 0), load redistributes to A and B (round 1), A exceeds tolerance and fails (round 2). Blast radius:  $2/4$  nodes = 50%.

**Algebraic connectivity.** The Fiedler value [51–53]  $\lambda_2$ —the second-smallest eigenvalue of the graph Laplacian—measures resilience to partitioning. A decrease in  $\lambda_2$  after applying changes indicates reduced robustness, consistent with the findings of Albert et al. [54] on the vulnerability of scale-free networks.

Figure 19 illustrates how removing a critical link reduces the Fiedler value, bringing the network closer to partitioning.



**Figure 19.** Spectral resilience: removing link B–E reduces the Fiedler value from 1.38 to 0.42 (a 70% decrease), indicating the network is approaching a partition boundary between  $\{A, B, C, D\}$  and  $\{E, F\}$ .

**Fragility heatmap.** `NETASSURE`’s `compute_fragility_heatmap()` function scores each node and edge by the cascade size that its failure would trigger. `NETSEQUENCE` uses this to identify *fragility hotspots*: network elements whose removal causes disproportionate cascade propagation. If a proposed mutation touches a fragility hotspot, the blast radius risk score is elevated and the change group is flagged for careful sequencing (e.g., make-before-break, or traffic pre-drainage).

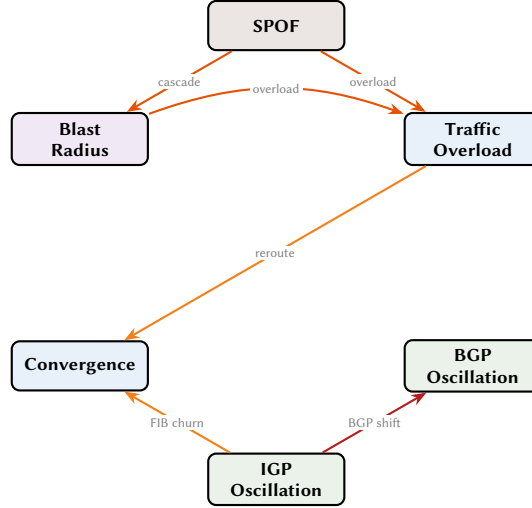
**Risk score.**

$$R_{\text{blast}} = w_1 \cdot \frac{\bar{C}_{\text{mc}}}{|V|} + w_2 \cdot \max\left(0, \frac{\lambda_{2,0} - \lambda_2^*}{\lambda_{2,0}}\right) + w_3 \cdot \Delta_{\text{eigengap}} \quad (8)$$

where  $\bar{C}_{\text{mc}}$  is the mean cascade size from Monte Carlo simulation and  $\Delta_{\text{eigengap}}$  is the change in spectral eigengap.

### 5.7 Cross-Domain Risk Interactions

The six risk domains are not fully independent: changes that elevate risk in one domain often affect others. Figure 20 maps the key interactions.



**Figure 20.** Cross-domain risk interactions. Key coupling: IGP metric changes can trigger BGP route shifts (the “cure worse than disease” effect [55]); SPOF changes cascade into both overload and blast radius; convergence is affected by both IGP and traffic domains.

The most critical interaction is between IGP and BGP: an IGP metric change that alters the shortest path can cause iBGP to select a different egress point, redistributing traffic across the network. `NETSEQUENCE` evaluates this by running the BGP analyzer on intermediate topologies that include pending IGP metric changes, catching the “cure worse than disease” phenomenon [55].

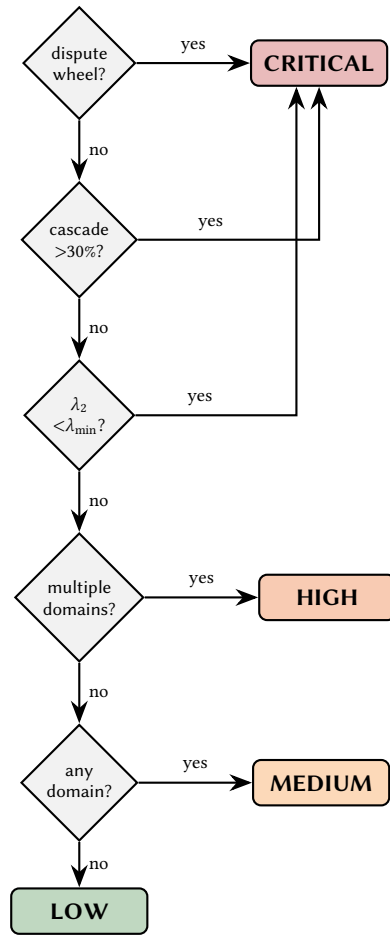
### 5.8 Risk Aggregation

Let  $\mathcal{D} = \{d_1, \dots, d_6\}$  denote the six risk domains and  $R_d(G, G') \in [0, 1]$  the normalised score for domain  $d$  on transition  $G \rightarrow G'$ . The aggregate risk score is the weighted sum

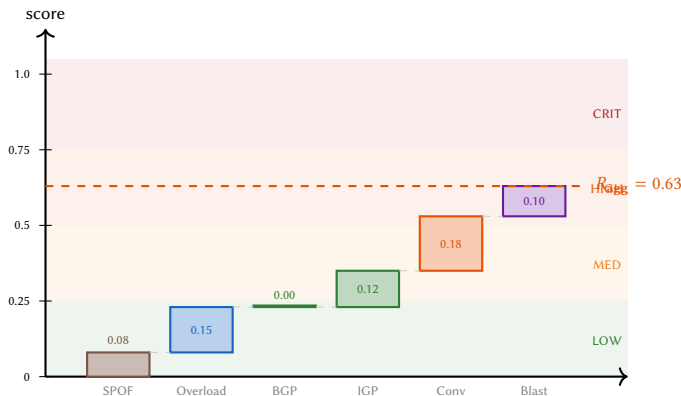
$$R_{\text{agg}}(G, G') = \sum_{d \in \mathcal{D}} w_d \cdot R_d(G, G'), \quad \text{where } \sum_d w_d = 1, \quad w_d > 0 \quad (9)$$

followed by a classification decision tree (fig. 21) that can promote the classification to CRITICAL independent of the weighted sum when structural invariants are violated (dispute wheels, catastrophic cascades, near-partition).

Figure 22 shows how the six weighted domain scores combine into the aggregate score for a concrete transition.



**Figure 21.** Risk aggregation decision tree. Checks cascade from top: any “yes” at the critical tier immediately classifies the change as CRITICAL.



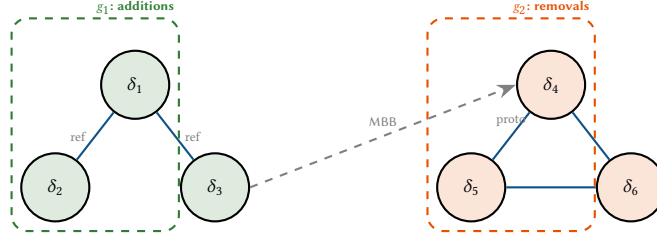
**Figure 22.** Risk score waterfall chart. Each domain’s score stacks cumulatively; the aggregate determines the classification zone. In this example, convergence risk is the dominant contributor, pushing the aggregate into the HIGH zone.

## 6 Change Sequencing

### 6.1 Group Construction

Mutations are clustered into atomic *change groups* using an affinity graph where deltas are connected if they share layer coupling (dependency-edge paths in the NTE graph), edge-node coupling (a REMOVE\_NODE and its incident edges), or protocol coupling (both ends of a protocol session). Connected components of the affinity graph yield the change groups.

Figure 23 illustrates affinity graph construction for the MutationPlan from fig. 3. The six mutations cluster into two groups: additions ( $\delta_1$ – $\delta_3$ ) and removals ( $\delta_4$ – $\delta_6$ ), connected by make-before-break dependencies.



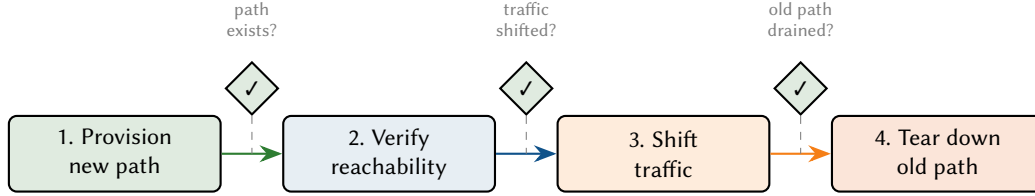
**Figure 23.** Affinity graph for the MutationPlan from fig. 3. Connected components form change groups  $g_1$  (additions) and  $g_2$  (removals). Edge labels: ref = reference integrity, proto = protocol coupling. The MBB dependency between groups ensures  $g_1$  executes before  $g_2$ .

## 6.2 Make-Before-Break

The make-before-break (MBB) constraint is central to safe network changes. When migrating traffic from path  $P_{old}$  to  $P_{new}$ , the new path must be fully established and carrying traffic before the old path is torn down. This prevents transient black-holes where neither path is available.

NETSEQUENCE detects MBB requirements by analysing reachability: for each source–destination pair  $(s, t)$  that uses edges in the removal set, it checks whether alternative paths exist in the intermediate topology. If not, it generates an MBB dependency edge in the DAG requiring the corresponding additions to complete first.

Figure 24 illustrates the four-phase MBB migration pattern that NETSEQUENCE generates.

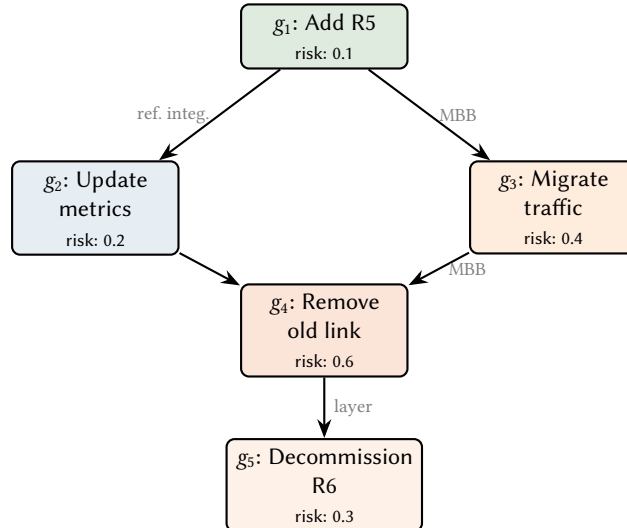


**Figure 24.** Make-before-break migration pattern with three verification gates. Each gate must pass before the next phase begins.

## 6.3 Dependency DAG

Change groups form a DAG based on three constraint types (fig. 25):

1. **Make-before-break:** new path established before old torn down.
2. **Layer ordering:** additions physical  $\rightarrow$  logical  $\rightarrow$  protocol  $\rightarrow$  service; removals reverse.
3. **Reference integrity:** edge additions follow endpoint node additions.



**Figure 25.** Dependency DAG for a five-group change plan. Edge labels show constraint type (MBB = make-before-break). Node colour intensity reflects risk score.

## 6.4 Risk-Weighted Topological Sort

The risk function for tie-breaking is the maximum domain score across all six analyzers for the transition that each group represents:

$$r(g) = \max_{d \in \mathcal{D}} R_d(G_{\text{before}(g)}, G_{\text{after}(g)}) \quad (10)$$

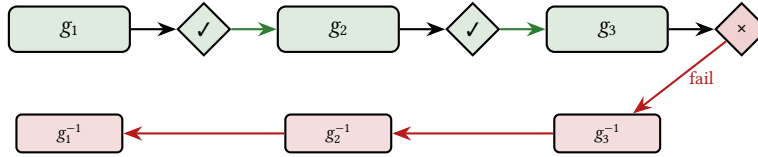
The sort algorithm (algorithm 2 in section A) uses Kahn’s algorithm [18] with a min-priority queue keyed by  $r(g)$ : at each step, the ready group with the lowest risk score is selected. This ensures low-risk changes are applied first, establishing a resilient state before higher-risk operations.

### 6.5 Execution with Verification Gates

Between consecutive groups, *verification gates* run a set of automated checks. Table 2 lists the checks performed at each gate, and fig. 26 shows the execution flow with rollback.

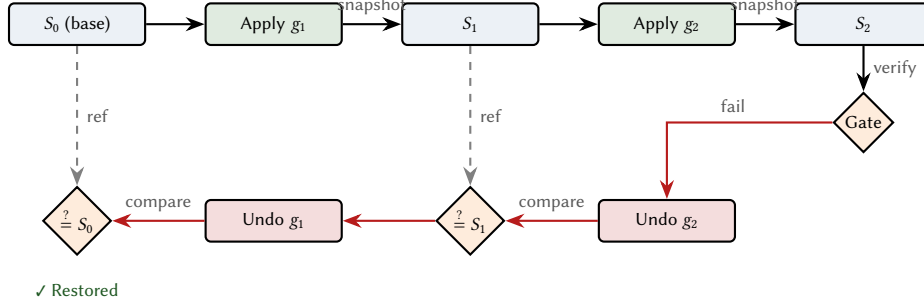
**Table 2.** Verification gate checks.

| Check        | Delegate          | Condition                                 |
|--------------|-------------------|-------------------------------------------|
| Reachability | netassure_verify  | All critical $(s, t)$ pairs reachable     |
| Loop-free    | netassure_verify  | No forwarding loops detected              |
| Convergence  | netsim            | FIB stable for $\geq 3$ consecutive ticks |
| Utilisation  | netflowsim        | All links below $\theta$ (default 80%)    |
| Spectral     | netassure_analyze | $\lambda_2 \geq \lambda_{\min}$           |



**Figure 26.** Execution flow with verification gates. Gate 3 fails, triggering reverse-order rollback:  $g_3^{-1}, g_2^{-1}, g_1^{-1}$ .

Figure 27 provides a more detailed view of the rollback mechanism, showing how MVCC snapshots enable precise reverse-order unwinding with intermediate verification.



**Figure 27.** Detailed rollback mechanism. Each forward step snapshots the topology state ( $S_0, S_1, S_2$ ). On gate failure, mutations are reversed in order; each intermediate state is compared against the stored MVCC snapshot to verify correctness.

## 7 Safe Network Transitions

A central challenge in network operations is ensuring that the *transition* between configurations is safe, not just the endpoints. This section surveys the landscape of safe update techniques and positions NETSEQUENCE’s approach within it.

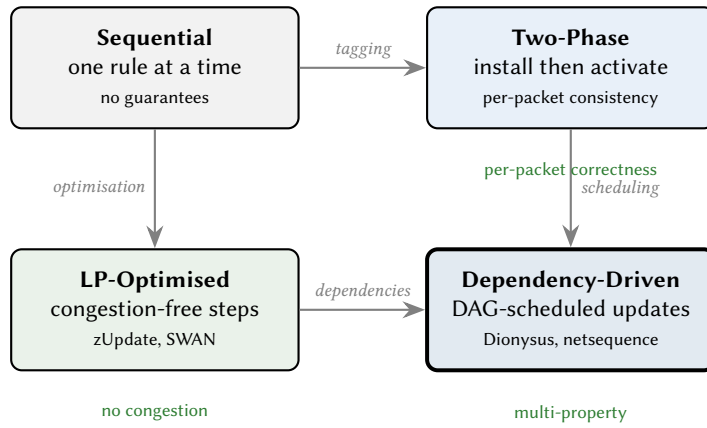
### 7.1 Update Paradigms

Figure 28 summarises the four main paradigms for safe network updates, progressing from simple sequential application to formally verified dependency-driven scheduling.

### 7.2 Consistent Network Updates

Reitblatt et al. [9] introduced the concept of *per-packet consistent updates*: every packet is processed entirely by the old configuration or entirely by the new one, never a mixture. This is achieved via a two-phase commit using packet tags (version stamps). While elegant, two-phase updates require switch support for tag-based forwarding and do not address *transient* properties like congestion or convergence time.

Katta et al. [56] extended this to *incremental consistent updates* that reduce the space overhead by updating one switch at a time while maintaining consistency. McClurg et al. [57] provided synthesis algorithms for computing minimal update sequences.



**Figure 28.** Safe network update paradigms. `NETSEQUENCE` operates in the dependency-driven paradigm, combining risk-weighted scheduling with multi-domain safety verification.

### 7.3 Congestion-Free Updates

zUpdate [11] addresses the problem of updating data centre WANs without causing transient congestion. It formulates the update scheduling as a linear program: given old and new traffic engineering configurations, find a sequence of intermediate configurations such that no link exceeds capacity at any step.

SWAN [58] takes a similar approach for inter-datacenter WANs, using multi-step traffic engineering updates with congestion-free intermediate states. Both systems demonstrate that congestion-free updates are achievable but require global coordination and LP solving, which may not scale to very large topologies.

### 7.4 Lossless IGP Migrations

Vanbever et al. [59, 60] studied the specific problem of migrating between IGP protocols (e.g., OSPF to IS-IS) or modifying IGP metrics without causing traffic loss. Their key insight is that “ships in the night” migration—running both IGPs simultaneously—can itself introduce transient loops if not carefully ordered. They compute a safe ordering of metric changes that avoids loops at every intermediate step.

Importantly, Vanbever et al. [55] showed that naïve graceful IGP operations can interact badly with BGP: the “cure” (gradual metric changes) can be “worse than the disease” (instantaneous cutover) because BGP reacts to each intermediate metric change, potentially causing traffic shifts and oscillations. This finding directly motivates `NETSEQUENCE`’s cross-domain risk analysis: the IGP oscillation analyzer alone is insufficient—BGP effects of IGP changes must also be evaluated.

### 7.5 Dependency-Driven Scheduling

Dionysus [10] is the closest prior work to `NETSEQUENCE`’s sequencing approach. It models network updates as a dependency graph where each update operation has preconditions (e.g., “install rule *A* before removing rule *B*”) and schedules them dynamically, adapting to real-time feedback from switches. Dionysus targets SDN environments with OpenFlow switches.

`NETSEQUENCE` extends this paradigm in three directions:

1. **Multi-domain risk:** Dionysus ensures reachability and loop-freedom; `NETSEQUENCE` additionally evaluates traffic overload, BGP oscillation, convergence time, and cascade risk.
2. **Distributed control planes:** Dionysus assumes centralised SDN control; `NETSEQUENCE` targets traditional distributed routing (OSPF, IS-IS, BGP) where update propagation is asynchronous and timing-dependent.
3. **Risk-weighted ordering:** Dionysus schedules for correctness; `NETSEQUENCE` additionally optimises for minimal risk exposure using the risk-weighted topological sort (section 6).

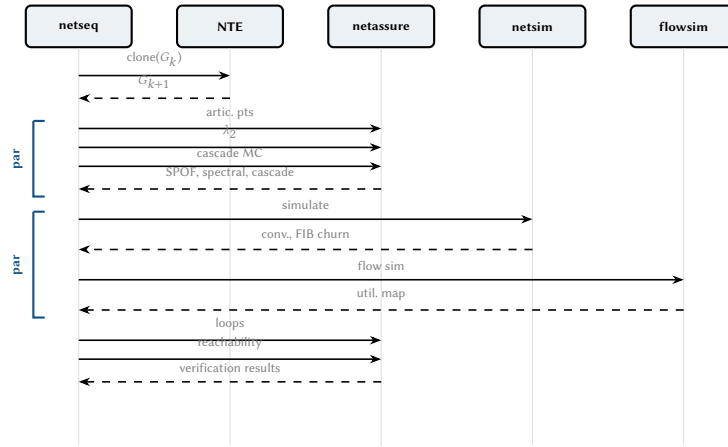
### 7.6 Network Synthesis

A complementary line of work synthesises correct-by-construction configurations. Propane [61] compiles high-level routing objectives into BGP configurations that are guaranteed to satisfy the objectives under all failure scenarios. NetComplete [62] uses SMT-based synthesis to auto-complete partial configurations. Bagpipe [63] verifies BGP configurations against formal specifications.

These tools address the *endpoint* problem (“is the target configuration correct?”) but not the *transition* problem (“how do we safely get there?”). `NETSEQUENCE` is complementary: it assumes the target state has been validated by such tools and focuses on the safe path from current to target.

## 8 Integration

NETSEQUENCE delegates all heavy computation to ecosystem crates. Figure 29 shows the interaction sequence for a single transition analysis, and table 3 summarises the full delegation map.



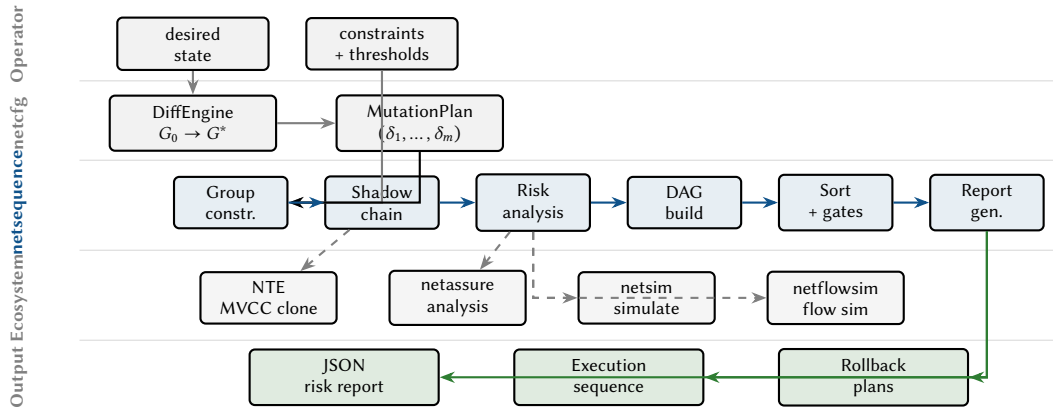
**Figure 29.** Integration sequence diagram for a single transition  $G_k \rightarrow G_{k+1}$ . Structural analysis, simulation, and flow analysis run in parallel via rayon.

Table 3 summarises the delegation map.

**Table 3.** Integration points between NETSEQUENCE and ecosystem crates.

| Concern          | Crate             | Key Entry Point                       |
|------------------|-------------------|---------------------------------------|
| Topology state   | netassure_model   | Topology, GraphDelta                  |
| Structural diff  | ank_netcfg        | DiffEngine                            |
| Robustness       | netassure_analyze | robustness::algebraic_connectivity()  |
| Path diversity   | netassure_analyze | paths::node_disjoint_paths()          |
| Spectral         | netassure_analyze | spectral::compute_spectral_clusters() |
| Centrality       | netassure_analyze | centrality::betweenness_centrality()  |
| Cascade sim.     | netassure_cascade | run_monte_carlo()                     |
| Fragility        | netassure_cascade | compute_fragility_heatmap()           |
| Loop detection   | netassure_verify  | detect_loops_on_graph()               |
| Reachability     | netassure_verify  | ReachabilityChecker                   |
| HSA              | netassure_verify  | HeaderSpace                           |
| Flow sim.        | netflowsim        | NetworkSimulator                      |
| Packet sim.      | network_simulator | Engine, Scenario                      |
| Convergence det. | network_simulator | Device::fib_changed()                 |

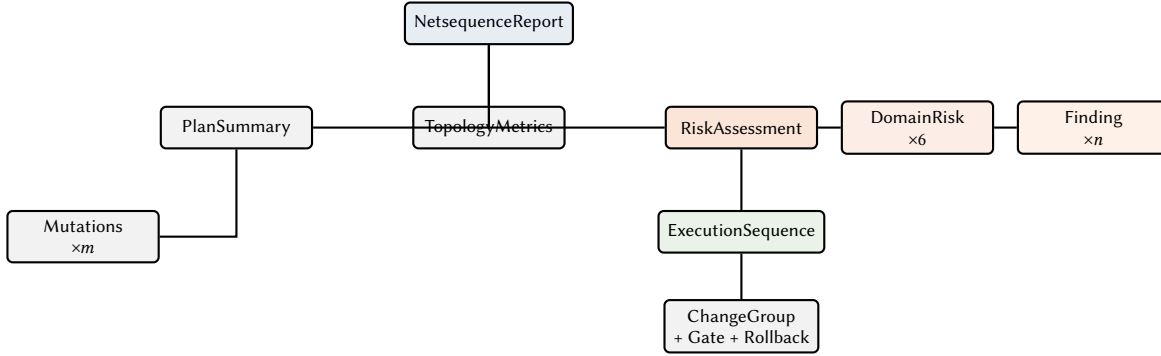
Figure 30 provides a high-level view of the complete data flow from operator input to report output.



**Figure 30.** End-to-end data flow swimlane diagram. The operator provides a desired state; NETCFG produces a MutationPlan; NETSEQUENCE orchestrates analysis via ecosystem crates and emits a structured report with execution sequence and rollback plans.

## 9 Report Schema

NETSEQUENCE produces a structured JSON report (fig. 31) containing plan summary, baseline topology metrics, per-domain risk scores with findings, aggregate classification, ordered execution sequence, and verification gate specifications.



**Figure 31.** Report schema structure. The JSON is consumed by downstream tooling (workbench, CI/CD pipelines, audit systems).

Figure 32 shows an abridged example of the JSON output for the worked example from section 26.

```

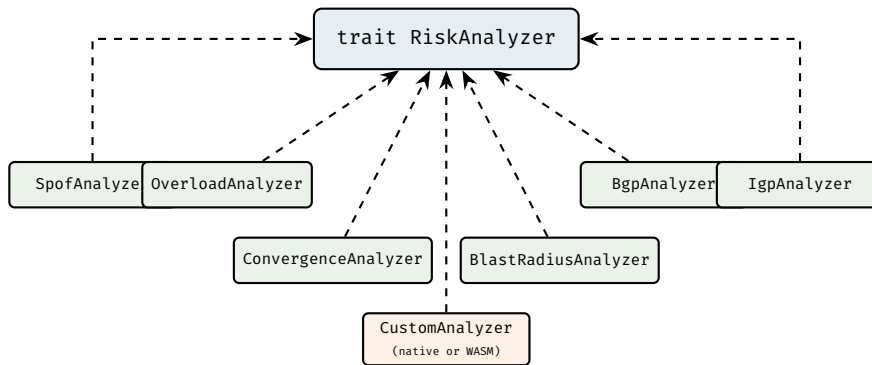
{
  "report_id": "NS-2026-03-001",
  "classification": "MEDIUM",
  "aggregate_score": 0.18,
  "topology": { "nodes": 12, "edges": 18, "fiedler": 1.82 },
  "risk_assessment": {
    "spof": { "score": 0.08, "new_aps": ["R5"], "bridges": 0 },
    "overload": { "score": 0.15, "peak_util": 0.72 },
    "bgp": { "score": 0.00, "dispute_wheels": 0 },
    "igp": { "score": 0.12, "micro_loop_prob": 0.08 },
    "convergence": { "score": 0.18, "t_conv_ms": 850 },
    "blast": { "score": 0.10, "fiedler_delta": -0.13 }
  },
  "execution_sequence": [
    { "group": "g1", "ops": ["AddNode(R5)", ...],
      "risk": 0.05, "gate": ["reachability", "loop_free"] },
    { "group": "g2", "ops": ["RemoveEdge(R1-R3)", ...],
      "risk": 0.18, "gate": ["reachability", "utilisation"] }
  ],
  "rollback_plans": [ { "g1_inv": [...], { "g2_inv": [...] } ]
}

```

**Figure 32.** Abridged JSON report output for the router replacement scenario. The full report includes per-finding details, rollback operations, and convergence timing data.

## 10 Extensibility

All risk analyzers implement a common `RiskAnalyzer` trait with methods for `name()`, `domain()`, and `analyze(baseline, proposed, plan)`. Analyzers register with an `AnalyzerRegistry` and run in parallel via `rayon`.



**Figure 33.** Analyzer trait hierarchy. Built-in and custom analyzers implement the same interface.

Future extensibility is planned via WASM plugins: each plugin is a `.wasm` module loaded at startup, running in a `wasmtime` sandbox with read-only access to serialised topology data and configurable resource limits.

## 11 Related Work

We situate `NETSEQUENCE` at the intersection of five research areas: network verification, BGP stability theory, network resilience, IGP convergence, and safe network updates.

### 11.1 Network Verification

Header Space Analysis (HSA) [6] introduced the idea of modelling forwarding behaviour as geometric operations on packet header spaces, enabling reachability and loop-free verification. NetPlumber [64] extended HSA to real-time incremental verification. VeriFlow [65] checks network-wide invariants in real time by slicing the equivalence classes affected by each rule update.

Batfish [5] parses vendor configurations and builds a data-plane model, enabling offline “what-if” analysis—similar in spirit to `NETSEQUENCE`’s shadow topologies, but operating at the configuration level rather than the topology-graph level. Minesweeper [7] applies SMT-based verification to find all possible violations of network-wide properties. CrystalNet [66] emulates large production networks at scale, ARC [67] computes an abstract representation of network forwarding for fast equivalence checking, and Tiramisu [13] extends verification to multi-layer network configurations.

`NETSEQUENCE` differs from these systems in that it focuses on *risk assessment and ordering* rather than correctness verification per se; it delegates verification to `netassure_verify` and focuses on the planning layer above.

### 11.2 BGP Stability Theory

The theoretical foundations for BGP convergence analysis were laid by Griffin, Shepherd, and Wilfong [28], who defined the Stable Paths Problem (SPP) and proved that convergence is guaranteed if and only if no dispute wheel exists. Their earlier work [31] introduced the Bad Gadget—a three-node topology with no stable path assignment—establishing that BGP divergence is a real concern even for simple topologies.

Gao and Rexford [29] showed that commercially-motivated policies (the valley-free condition) guarantee convergence, providing a practical sufficient condition. Sobrinho [30, 34] generalised these results into an algebraic framework: convergence is guaranteed when the routing algebra is *strictly monotone*. This provides the most general tool for reasoning about BGP convergence under arbitrary policy configurations.

MED-induced oscillations represent a distinct failure mode [32]. Griffin and Wilfong [33] showed that even with correct iBGP configuration, MED comparison can cause persistent oscillation because MED violates the independence of irrelevant alternatives. Cittadini et al. [35] documented how route redistribution between IGP and BGP—a common operational practice—can create policy violations equivalent to dispute wheels.

Feamster and Balakrishnan [36] developed `rcc`, a tool that detects BGP configuration faults through static analysis. Feamster and Rexford [68] built predictive models of BGP route selection, enabling network-wide prediction of BGP behaviour from router configurations.

Labovitz et al. [1] characterised BGP convergence delays in practice, showing that path exploration can lead to minutes-long convergence times. This motivates `NETSEQUENCE`’s separation of BGP and IGP oscillation analysis into distinct domains with distinct risk models.

### 11.3 Network Resilience and Cascade Failures

Albert, Jeong, and Barabási [54] demonstrated that scale-free networks are resilient to random failures but vulnerable to targeted attacks on high-degree nodes. The Motter-Lai model [49] formalised cascade propagation via betweenness-based load redistribution, providing the algorithmic foundation for `NETSEQUENCE`’s blast radius analysis.

Buldyrev et al. [69] extended cascade analysis to interdependent networks, showing that cross-layer dependencies amplify failure propagation—directly relevant to `NETSEQUENCE`’s multi-layer model where physical failures cascade through logical and protocol layers. Watts [70] identified the conditions under which small perturbations trigger network-wide cascades. Callaway et al. [71] and Cohen et al. [72] characterised percolation thresholds for random graph robustness.

Fiedler’s algebraic connectivity [51] and spectral graph theory [53] provide the mathematical basis for `NETSEQUENCE`’s resilience metrics. Sterbenz et al. [73] provide a comprehensive framework for network resilience encompassing survivability, disruption tolerance, and performability.

### 11.4 IGP Convergence and Fast Reroute

Zinin [37] formally analysed micro-loop formation during IGP convergence. François and Bonaventure [39] showed that ordered FIB updates eliminate micro-loops; François’s thesis [40] provides the complete theoretical treatment including loop-free convergence conditions. RFC 8333 [38] standardised local convergence delay as a practical mitigation, and RFC 5715 [48] formalises the loop-free convergence framework.

The IP Fast Reroute family [42] provides sub-50 ms protection. LFA [44] offers per-prefix backup next-hops; Remote LFA [45] extends coverage via targeted LDP tunnels; and TI-LFA [43] achieves 100% coverage using

Segment Routing [46]. MPLS Fast Reroute [74] protects label-switched paths with facility backup or one-to-one backup tunnels.

Basu and Riecke [41] analysed OSPF stability, showing that route flap damping and SPF scheduling interact in complex ways. Fortz and Thorup [75] developed methods for optimising IGP weights under changing traffic matrices—relevant to `NETSEQUENCE`’s traffic overload analysis during metric changes.

### 11.5 Safe Network Updates

The safe network update problem has received substantial attention. Reitblatt et al. [9] introduced per-packet consistent updates using version stamps. Katta et al. [56] reduced space overhead with incremental consistent updates, and McClurg et al. [57] synthesised minimal update sequences.

For traffic engineering, `zUpdate` [11] computes congestion-free update sequences via LP, `SWAN` [58] achieves high WAN utilisation with multi-step congestion-free transitions, and Google’s B4 [76] demonstrates the scale at which such update mechanisms must operate. Dionysus [10] introduced dependency-driven dynamic scheduling of network updates, the closest prior work to `NETSEQUENCE`’s sequencing approach (see section 7 for a detailed comparison).

Vanbever et al. [59, 60] solved the lossless IGP migration problem, computing safe orderings of metric changes. Crucially, Vanbever et al. [55] showed that graceful IGP operations can cause BGP disruption—the “cure worse than the disease” phenomenon—motivating `NETSEQUENCE`’s cross-domain analysis. Vissicchio et al. [77] extended safe reconfiguration to scenarios involving route redistribution between protocols.

On the synthesis side, `Propane` [61] compiles routing objectives into correct-by-construction BGP configurations, `NetComplete` [62] auto-completes partial configurations via SMT, and `Bagpipe` [63] verifies BGP configurations against specifications. These address the endpoint problem; `NETSEQUENCE` addresses the transition.

### 11.6 Network Change Management

Doyle et al. [78] characterised the “robust yet fragile” nature of complex engineered systems, arguing that optimisation for one class of perturbation can increase vulnerability to others—a principle that motivates `NETSEQUENCE`’s multi-domain risk assessment. Markopoulou et al. [2] characterised failures in IP backbone networks, finding that planned maintenance events contribute significantly to disruption when poorly ordered.

Mahajan et al. [79] quantified BGP misconfiguration prevalence, showing that origin misconfigurations and export policy errors are responsible for a large fraction of Internet routing incidents. Gill et al. [80] studied data centre network failures and found that network device failures are the single largest contributor to service outages, with failures often clustering across redundancy boundaries. Potharaju and Jain [81] extended this analysis to middlebox failures, finding that maintenance-induced failures account for a significant fraction of incidents.

Benson et al. [82] measured the operational complexity of real networks and found that complexity—not size—is the primary driver of configuration errors. This motivates `NETSEQUENCE`’s approach of automating risk analysis rather than relying on operator expertise.

### 11.7 Network Digital Twins

The concept of a *network digital twin*—a virtual replica of the production network for safe experimentation—is closely related to `NETSEQUENCE`’s shadow topology approach. Almasan et al. [83] surveyed the enabling technologies and research challenges for network digital twins. RFC 8345 [12] defines a YANG data model for network topologies that provides a standardised representation compatible with digital twin construction.

`NETSEQUENCE`’s MVCC shadow topologies are a lightweight form of digital twin: rather than emulating the full network, they capture the graph structure and run analytical (not emulation-based) risk assessment. This trades fidelity for speed, enabling interactive planning workflows that would be impractical with full-fidelity emulation.

## 12 Naive vs. Intelligent Execution

To motivate `NETSEQUENCE`’s value, we contrast naïve (unplanned) execution of a change plan with the intelligent sequencing that `NETSEQUENCE` produces. We use a realistic scenario: a 15-node ISP backbone spanning the US Eastern seaboard, undergoing three simultaneous changes during a maintenance window.

### 12.1 Scenario: Backbone Maintenance Window

The network spans 15 points of presence (PoPs) including New York (NYK), Chicago (CHI), Atlanta (ATL), Dallas (DAL), Washington DC (DCA), and Denver (DEN). The operator has three tasks:

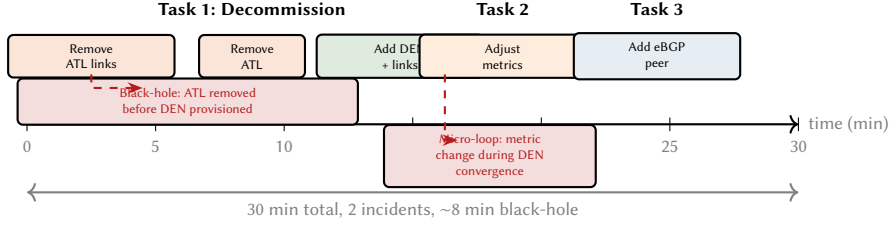
1. Decommission the ATL router (replace with a new DEN router on a different path).

2. Adjust OSPF metrics on the CHI–DCA and DCA–DAL links (traffic engineering).
3. Add a new eBGP peering session on NYK.

The MutationPlan contains 14 operations across these three tasks.

## 12.2 Naïve Execution

Without NETSEQUENCE, an operator might execute changes in task order (decommission first, then metrics, then BGP). Figure 34 shows the resulting timeline and incidents.



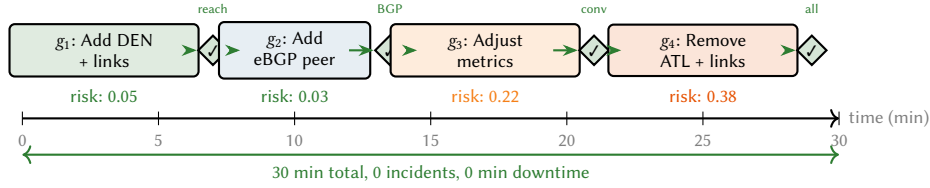
**Figure 34.** Naïve execution timeline. The ATL router is removed before DEN is provisioned, creating an 8-minute black-hole. Metric changes during DEN’s convergence window cause micro-loops.

The naïve approach causes two incidents:

1. **Black-hole (8 min):** ATL’s links are removed at  $t = 2$  but DEN is not provisioned until  $t = 10$ . All traffic transiting ATL is dropped.
2. **Micro-loop (45 sec):** OSPF metric changes on CHI–DCA at  $t = 17$  overlap with DEN’s convergence window, causing transient forwarding loops between CHI and DCA.

## 12.3 Intelligent Execution

NETSEQUENCE analyses the 14 mutations and produces the execution shown in fig. 35.



**Figure 35.** Intelligent execution timeline produced by NETSEQUENCE. Changes are reordered: low-risk additions (DEN) first, verification gates between each group, high-risk removals (ATL) last. Zero incidents.

NETSEQUENCE reorders the operations:

1.  $g_1$ : **Add DEN + links** (risk 0.05)—establish the replacement path first. Gate verifies reachability via DEN.
2.  $g_2$ : **Add eBGP peer on NYK** (risk 0.03)—independent of ATL decommission, safe to apply early.
3.  $g_3$ : **Adjust OSPF metrics** (risk 0.22)—wait until DEN has fully converged, avoiding metric/convergence interaction.
4.  $g_4$ : **Remove ATL + links** (risk 0.38)—highest risk, executed last when all alternatives are established.

## 12.4 Step-by-Step State Sequence

Figure 36 shows the network topology at each stage of the intelligent execution plan. Each panel represents the topology  $G_k$  after applying change group  $g_k$ , with the verification gate result annotating the transition. This visualisation is the primary output artifact of NETSEQUENCE: a concrete, auditable record of how the network evolves through each step.

The state sequence in fig. 36 illustrates two key principles of the NETSEQUENCE approach. First, *make-before-break*: alternative capacity (DEN) is provisioned in  $g_1$  before the existing capacity (ATL) is removed in  $g_4$ . Second, *gate-and-proceed*: every intermediate state is verified against the safety invariants before the next group is applied, ensuring that no transition leaves the network in a degraded state.

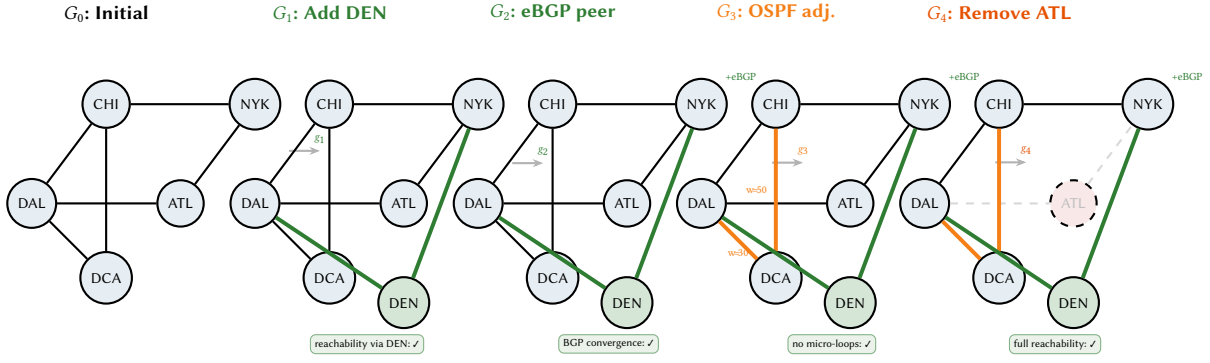
## 12.5 Comparison

Figure 37 provides a side-by-side comparison of the network state at the critical moment ( $t = 7$  min) under both approaches.

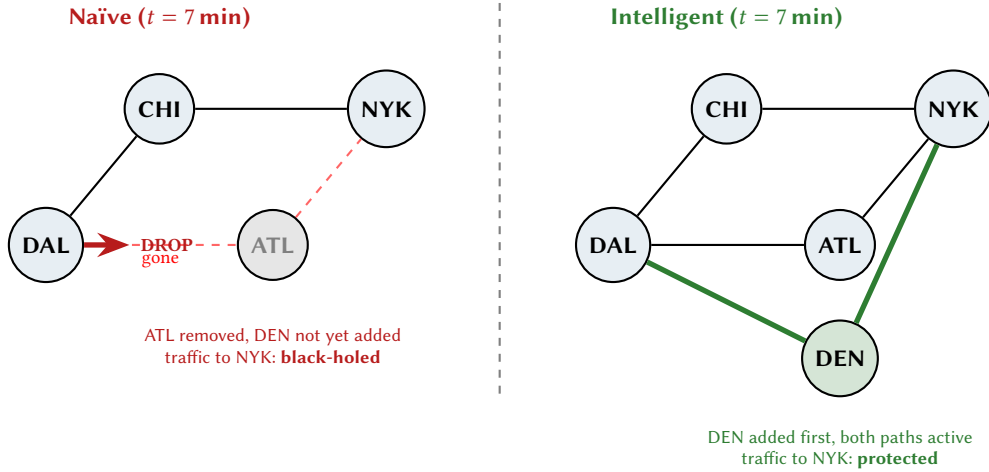
Table 4 summarises the quantitative differences.

## 12.6 Risk Profile Visualisation

NETSEQUENCE produces a per-transition risk profile showing all six domain scores. Figure 38 compares the risk profiles for the naïve and intelligent orderings at their respective highest-risk transitions.



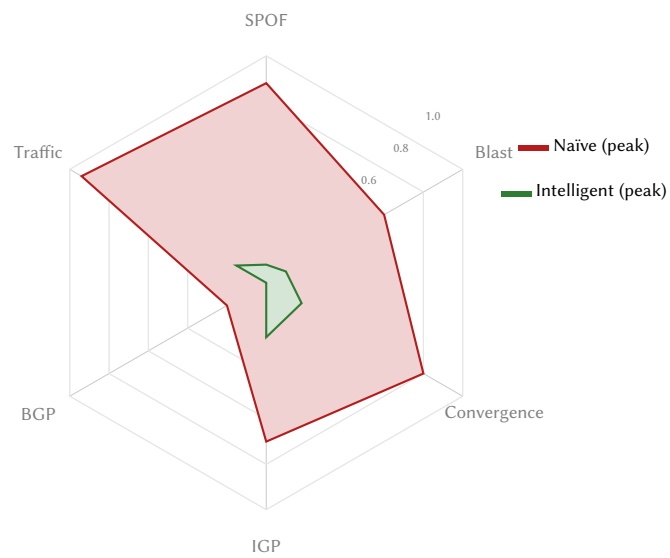
**Figure 36.** Step-by-step topology evolution under intelligent execution. Each panel shows state  $G_k$  after applying group  $g_k$ . Green nodes/links are additions; dashed grey elements are pending removal. Gate results appear below each state. Alternative paths (DEN) are established before the high-risk removal (ATL), and each intermediate state is validated before proceeding.



**Figure 37.** Network state at  $t = 7$  min. **Left:** naïve execution removed ATL before adding DEN, creating a black-hole. **Right:** intelligent execution added DEN first; both paths are active and traffic is protected.

**Table 4.** Naïve vs. intelligent execution comparison.

| Metric                | Naïve           | Intelligent |
|-----------------------|-----------------|-------------|
| Total duration        | 30 min          | 30 min      |
| Black-hole duration   | 8 min           | 0           |
| Micro-loop incidents  | 1 (45 s)        | 0           |
| Peak link utilisation | 94%             | 72%         |
| Min. path diversity   | 0 (partitioned) | 2           |
| $\lambda_2$ minimum   | 0.31            | 1.52        |
| Verification gates    | 0               | 4           |
| Rollback capability   | none            | full        |



**Figure 38.** Risk radar chart comparing peak-transition risk profiles. Naïve execution (red) shows critical risk in SPOF, traffic, and convergence domains. Intelligent execution (green) keeps all domains well within acceptable bounds.

## 13 Scenario: Data Centre Fabric Expansion

To illustrate **NETSEQUENCE** on a different topology type, we consider expanding a leaf-spine data centre fabric by adding a new spine switch and rebalancing ECMP.

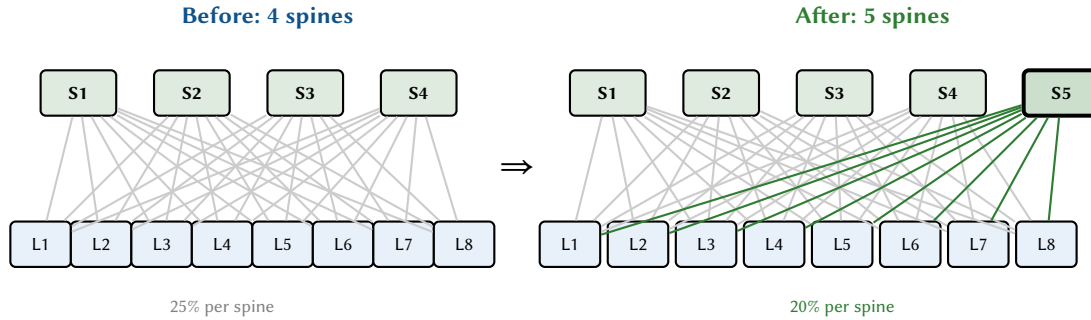
### 13.1 Baseline Topology

The fabric has 4 spine switches (S1–S4), 8 leaf switches (L1–L8), and full-mesh connectivity between each leaf and all spines (32 links). Each spine carries approximately 25% of east-west traffic.

### 13.2 Change Plan

Add spine S5 with links to all 8 leaves, then adjust ECMP weights to rebalance traffic across 5 spines (target: 20% each).

Figure 39 shows the topology before and after.



**Figure 39.** Leaf-spine fabric expansion: adding spine S5 and rebalancing ECMP from 4-way to 5-way distribution.

### 13.3 Risk Analysis

**NETSEQUENCE** identifies two key risks:

1. **ECMP polarisation:** during the transition where S5 has links to only some leaves, traffic hashing may overload the partially-connected spine. Risk: add all S5–leaf links atomically (single change group).
2. **Transient asymmetry:** if ECMP weights are adjusted before all links are active, some leaves will hash traffic to a non-existent S5 path. Risk: provision links before weight adjustment.

The generated execution sequence:

1.  $g_1$ : Add S5 node + all 8 links (atomic).
2. Gate: verify all  $L_i$ –S5 adjacencies established.
3.  $g_2$ : Adjust ECMP weights to 5-way.
4. Gate: verify traffic distribution within 5% of target.

## 14 Scenario: IGP Metric Migration

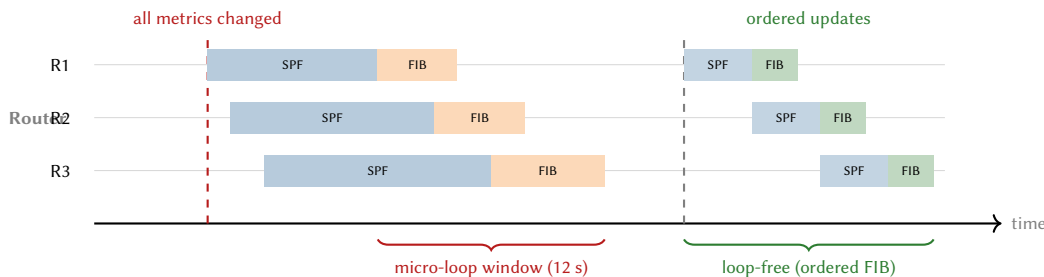
This scenario demonstrates how **NETSEQUENCE** handles the subtle interaction between IGP metric changes and BGP behaviour documented by Vanbever et al. [55].

### 14.1 Problem

An operator wants to change OSPF link metrics across a 20-router network to shift traffic from a congested core path to an under-utilised alternate path. The change involves modifying metrics on 6 links.

### 14.2 Naïve Approach

Changing all 6 metrics simultaneously triggers 6 overlapping SPF recomputations. Each router receives LSAs in unpredictable order, creating a 12-second window of micro-loops (fig. 40).



**Figure 40.** IGP metric migration: naïve simultaneous update (left) creates a 12-second micro-loop window. Ordered FIB updates (right) eliminate loops entirely.

### 14.3 Intelligent Approach

NETSEQUENCE applies three strategies:

1. **Ordered FIB updates:** computes a safe update ordering [39] so that routers are updated in an order that prevents loops at every intermediate step.
2. **Metric batching:** groups the 6 metric changes into 2 batches based on geographic proximity, reducing the number of concurrent SPF recomputations.
3. **BGP impact assessment:** checks whether the metric changes will alter iBGP next-hop selection (the “cure worse than disease” scenario [55]). If so, the metric change groups are further subdivided to avoid BGP route flapping.

## 15 Scenario: BGP Peering Policy Change

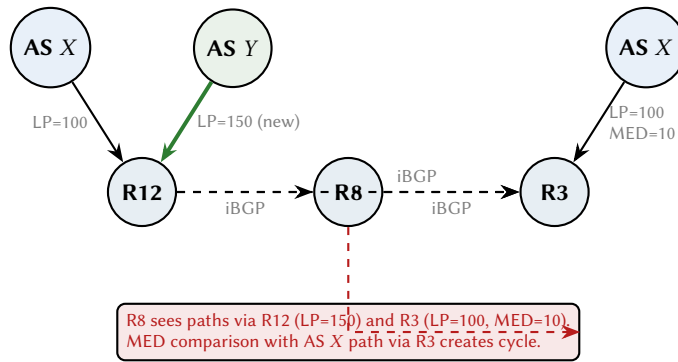
This scenario demonstrates how NETSEQUENCE detects potential BGP oscillation introduced by a policy change, a risk that is invisible to tools that only check reachability.

### 15.1 Problem

An operator wants to change the BGP community-based filtering on router R12 to prefer a new transit provider (AS Y) over the existing provider (AS X) for a set of 500 prefixes. The change involves modifying the local-preference on the eBGP import policy.

### 15.2 Risk Detection

NETSEQUENCE’s BGP oscillation analyzer extracts the policy graph and discovers that the new local-preference ranking, combined with an existing MED-based tie-breaking rule on router R8, creates a non-monotonic preference cycle (fig. 41).



**Figure 41.** BGP policy change scenario. Raising local-preference for AS Y on R12 interacts with MED-based selection on R8, where paths from AS X via R3 compete. NETSEQUENCE detects the potential MED oscillation before deployment.

### 15.3 Mitigation

NETSEQUENCE classifies this plan as CRITICAL (dispute wheel detected) and recommends:

1. Disable MED comparison on R8 for the affected prefix set before applying the local-preference change.
2. Alternatively, apply the local-preference change with always-compare-med enabled network-wide, which restores monotonicity [30].

Without NETSEQUENCE, this oscillation would only manifest in production after the policy is deployed, potentially affecting all 500 prefixes.

## 16 Scenario: Dual-Homed Customer Migration

This scenario illustrates a multi-layer change: migrating a dual-homed customer from one pair of PE routers to another, involving physical, logical, protocol, and service layer changes simultaneously.

### 16.1 Topology

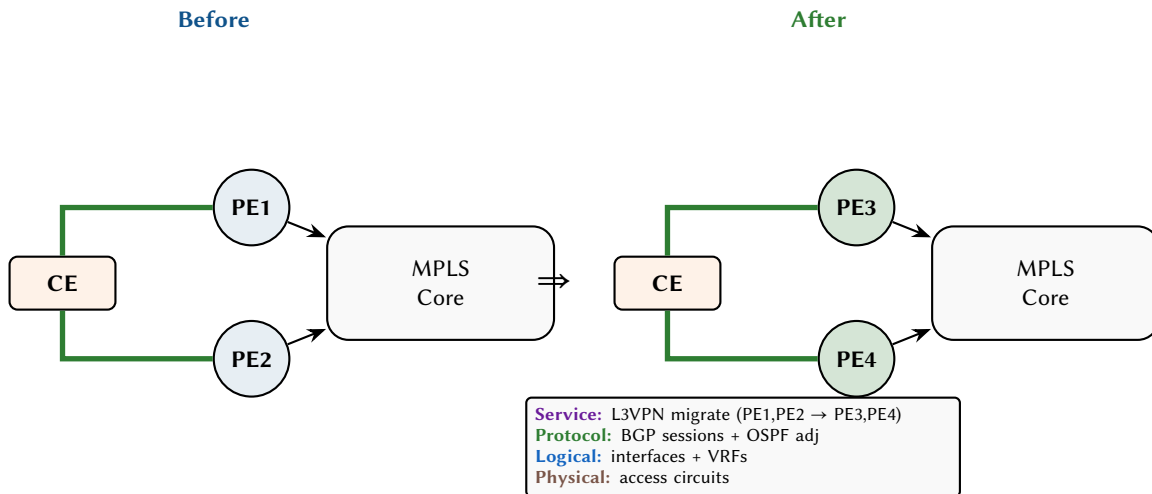
Customer C is connected to PE1 and PE2 via two access links. The target state moves the customer to PE3 and PE4 (a different PoP). The migration involves 12 mutations across all four topology layers (fig. 42).

### 16.2 Sequencing Challenge

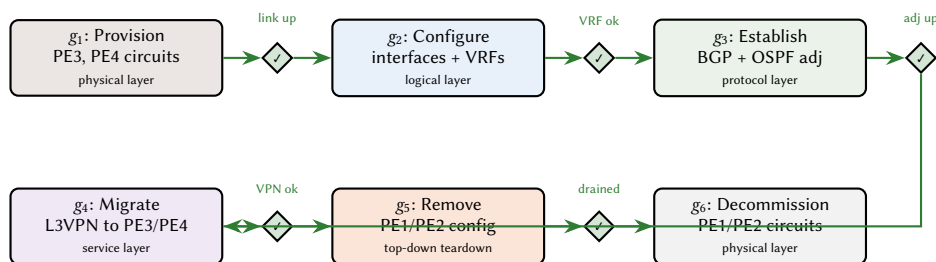
The key constraint is that the customer must maintain at least one active connection throughout the migration. NETSEQUENCE generates the following execution sequence:

During phases  $g_3$ – $g_5$ , the customer is simultaneously connected to both old (PE1/PE2) and new (PE3/PE4) routers, providing hitless migration. NETSEQUENCE’s risk analysis confirms:

- **SPOF:** no single point of failure at any intermediate state (customer always has  $\geq 2$  active paths).



**Figure 42.** Dual-homed customer migration across PoPs. The change spans all four topology layers and requires careful sequencing to maintain service continuity.



**Figure 43.** Customer migration execution sequence. Additions proceed bottom-up through layers ( $g_1$ – $g_4$ ); removals proceed top-down ( $g_5$ – $g_6$ ). Orthogonal routing connects the two phases via Gate 3. The customer has dual-homing throughout  $g_3$ – $g_5$ .

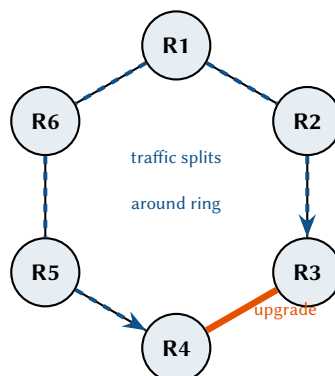
- **Traffic:** ECMP across 4 PEs during the overlap window keeps utilisation below 50%.
- **Blast radius:** Fiedler value increases during the overlap (more connectivity), confirming the sequencing is safe.

## 17 Scenario: Ring Network Link Upgrade

This scenario demonstrates how **NETSEQUENCE** handles a common operational task—upgrading a link in a ring topology—where the naive approach creates a single point of failure.

### 17.1 Topology

A 6-node ring (R1–R2–R3–R4–R5–R6–R1) carries bidirectional traffic using OSPF with equal metrics. The operator wants to upgrade link R3–R4 from 10G to 100G, which requires a brief link-down period.



**Figure 44.** Ring topology with link R3–R4 targeted for upgrade. During the upgrade, all traffic must traverse the long path (R3–R2–R1–R6–R5–R4).

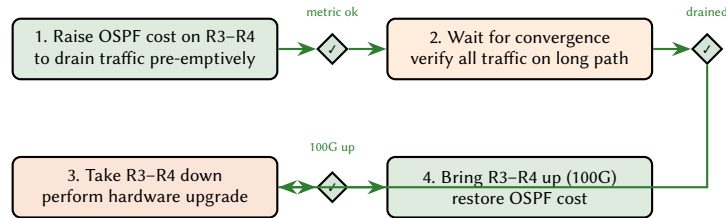
### 17.2 Risk Analysis

When R3–R4 goes down for the upgrade, the ring becomes a chain. **NETSEQUENCE** identifies:

- **SPOF**: every node on the chain becomes an articulation point. In particular, R1 and R6 (the “middle” of the surviving path) become critical.
- **Traffic overload**: all traffic that previously used R3–R4 shifts to the long path. If the ring was 60% utilised, the surviving path reaches 120% of its per-link capacity—guaranteed overload.
- **Convergence**: OSPF reconvergence causes a ~1-second micro-loop window as R3 and R4 recompute their FIBs.

### 17.3 Intelligent Approach

NETSEQUENCE generates a four-step plan:



**Figure 45.** Ring link upgrade execution sequence. Traffic is drained *before* the link is taken down, avoiding the transient overload that a naïve approach would cause. Orthogonal routing connects the drain and upgrade phases.

The critical insight is step 1: by pre-emptively draining traffic via an OSPF metric increase, the actual link-down event (step 3) causes zero additional traffic shift. The micro-loop window is also eliminated because all traffic has already converged to the long path before the link goes down.

### 17.4 Quantitative Comparison

**Table 5.** Ring link upgrade: naïve vs. intelligent.

| Metric              | Naïve             | Intelligent             |
|---------------------|-------------------|-------------------------|
| Traffic disruption  | ~1 s (micro-loop) | 0                       |
| Peak utilisation    | 120% (overload)   | 95% (pre-drained)       |
| Articulation points | 4 (surprise)      | 4 (expected, monitored) |
| Rollback available  | no                | yes (restore metric)    |

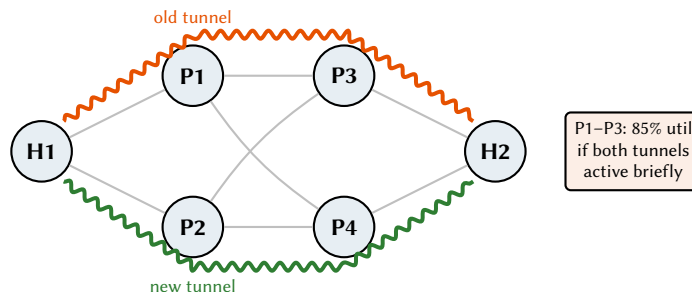
## 18 Scenario: MPLS TE Tunnel Re-Optimisation

This scenario shows how NETSEQUENCE handles changes to the traffic engineering overlay, where the interaction between the TE tunnels and the underlying IGP creates subtle risks.

### 18.1 Problem

The operator needs to re-optimize 8 MPLS TE tunnels across a 30-router network to accommodate a new traffic matrix. The tunnels carry 70% of inter-site traffic. Re-optimisation involves modifying tunnel paths (explicit route objects) and bandwidth reservations.

### 18.2 Risk Analysis



**Figure 46.** MPLS TE tunnel re-optimisation. Moving the tunnel from the top path to the bottom path. If both tunnels are active simultaneously, link P1–P3 is overloaded.

NETSEQUENCE identifies that naïvely signalling all 8 new tunnels before tearing down old ones would transiently double-book bandwidth on shared links. The risk-weighted sort produces a per-tunnel interleaved schedule:

1. Group tunnels by shared physical links.
2. For each group: signal new tunnel → verify → tear down old tunnel → verify.

- Order groups by bandwidth: largest tunnels first (highest overload risk if mis-sequenced).

This make-before-break-per-tunnel approach keeps peak link utilisation below 80% throughout the re-optimisation, compared to 140% peak under naïve bulk re-signalling.

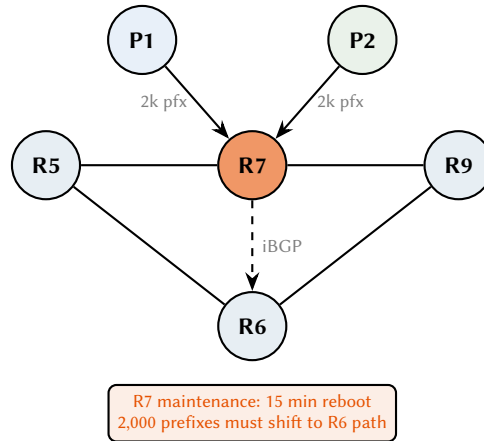
## 19 Scenario: Graceful BGP Shutdown

RFC 8326 [84] defines the `GRACEFUL_SHUTDOWN` community for draining traffic from a BGP session before maintenance. This scenario shows how `NETSEQUENCE` integrates graceful shutdown into its sequencing, and how naïve application can still cause problems.

### 19.1 Problem

An operator needs to perform maintenance on router R7, which has two eBGP peers (upstream providers P1 and P2) and carries transit traffic for 2,000 prefixes. The maintenance requires a 15-minute reboot.

### 19.2 Topology



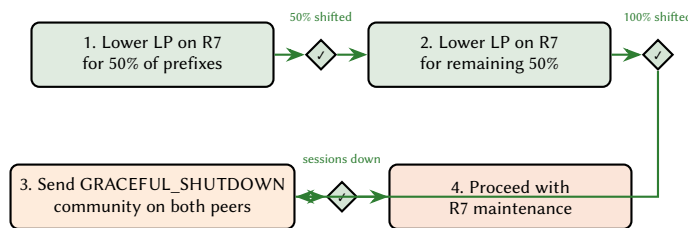
**Figure 47.** Graceful BGP shutdown scenario. R7 must be drained before maintenance. Traffic for 2,000 prefixes needs to shift to the R6 alternate path via P1/P2’s other ingress points.

### 19.3 Naïve Approach

Simply sending `GRACEFUL_SHUTDOWN` on both sessions simultaneously causes all 2,000 prefixes to withdraw from R7 at once. The resulting BGP convergence storm takes 45 seconds as all downstream routers recompute their best paths. During this window, some prefixes are transiently unreachable.

### 19.4 Intelligent Approach

`NETSEQUENCE` generates a four-phase plan:



**Figure 48.** Graceful shutdown execution sequence. Traffic is drained in two batches to limit convergence storm magnitude, then formal graceful shutdown is applied before maintenance.

By batching the prefix drain, `NETSEQUENCE` limits the convergence storm to 1,000 prefixes at a time. The verification gate after each batch confirms that the alternate path via R6 has absorbed the traffic without exceeding the 80% utilisation threshold. The quantitative impact:

**Table 6.** Graceful shutdown: naïve vs. intelligent.

| Metric                | Naïve            | Intelligent              |
|-----------------------|------------------|--------------------------|
| Convergence storm     | 2,000 pfx (45 s) | 1,000 pfx ×2 (12 s each) |
| Peak BGP churn        | 4,000 updates/s  | 2,100 updates/s          |
| Transient unreachable | 12 prefixes      | 0                        |
| Utilisation spike     | 88%              | 76% (batched)            |

## 20 Scenario: EVPN/VXLAN Fabric Migration

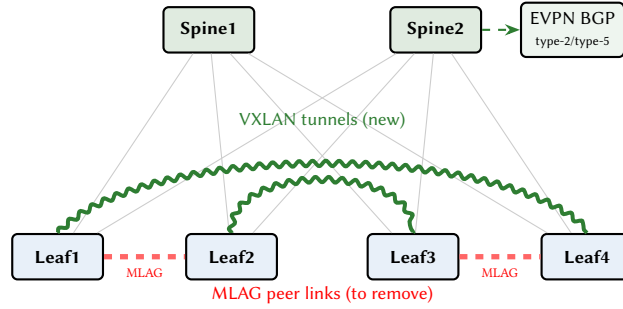
This scenario illustrates a data centre fabric migration from traditional VLAN-based L2 to EVPN/VXLAN overlay [85, 86], a multi-phase change involving control plane, data plane, and service layer modifications.

### 20.1 Problem

A 24-switch leaf-spine fabric currently uses MLAG-based L2 extension for inter-rack connectivity. The target architecture replaces MLAG with EVPN-VXLAN, which requires:

1. Enable VXLAN tunnelling on all leaf switches.
2. Configure EVPN BGP address family on all leaf-spine sessions.
3. Migrate each VLAN to its corresponding VNI.
4. Decommission MLAG peer links.

### 20.2 Risk Analysis



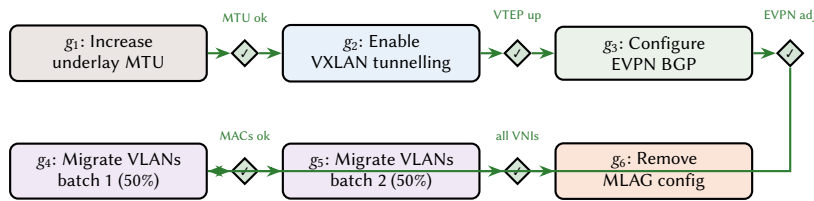
**Figure 49.** EVPN/VXLAN fabric migration. MLAG peer links (dashed red) are replaced by VXLAN overlay tunnels (green). EVPN BGP provides the control plane for MAC/IP learning.

NETSEQUENCE identifies four critical risks:

1. **BFD interaction** [87]: enabling VXLAN changes the BFD session topology, potentially causing false failure detection during migration.
2. **MAC learning storm**: during the transition where both MLAG and EVPN are active for the same VLAN, duplicate MAC advertisements can cause learning oscillations.
3. **ARP flooding**: EVPN type-2 routes may not be fully populated when the first VLANs migrate, causing ARP broadcast storms on the VXLAN overlay.
4. **MTU mismatch**: VXLAN encapsulation adds 50 bytes; if the underlay MTU is not increased first, jumbo frames will be silently dropped.

### 20.3 Execution Sequence

NETSEQUENCE generates an 8-group execution plan:



**Figure 50.** EVPN/VXLAN migration execution sequence. Infrastructure changes ( $g_1$ – $g_3$ ) precede VLAN migration ( $g_4$ – $g_5$ ); MLAG teardown ( $g_6$ ) is last. VLANs are migrated in batches to limit MAC learning storm scope.

The critical insight is the MTU-first ordering ( $g_1$ ): without increasing underlay MTU before enabling VXLAN, the 50-byte encapsulation overhead would cause silent drops for any 1500-byte frames. This is a cross-layer dependency that only NETSEQUENCE’s multi-layer model captures.

## 21 Formal Safety Properties

We formalise the safety guarantees that NETSEQUENCE’s sequencing algorithm provides. Throughout, let  $\text{apply}(G, g)$  denote the topology obtained by applying change group  $g$  to topology  $G$ .

**Definition 21.1 (Safe Transition).** A transition  $G_k \xrightarrow{g_{k+1}} G_{k+1}$  is safe if:

1. the aggregate risk score satisfies  $R_{\text{agg}}(G_k, G_{k+1}) \leq \tau$  for operator-specified threshold  $\tau$ , and
2. all verification gate predicates hold:  $\bigwedge_{p \in \mathcal{P}} p(G_{k+1}) = \top$ .

**Definition 21.2 (Safe Execution Sequence).** An execution sequence  $S = [g_{\pi(1)}, \dots, g_{\pi(n)}]$  is safe if every transition is safe:  $\forall k \in [1, n] : R_{agg}(G_{k-1}, G_k) \leq \tau \wedge \bigwedge_{p \in \mathcal{P}} p(G_k) = \top$ .

**Definition 21.3 (Invertibility).** A change group  $g$  is invertible on  $G$  if there exists  $g^{-1}$  such that applying  $g$  then  $g^{-1}$  recovers  $G$ :  $\text{apply}(\text{apply}(G, g), g^{-1}) = G$ .

**Lemma 21.1 (Group Invertibility).** Every change group  $g_k$  produced by algorithm 1 is invertible on  $G_{k-1}$ , with the inverse given by algorithm 3.

*Proof.* Each primitive mutation has a well-defined inverse:  $\text{ADDNODE}(n, \ell, a)^{-1} = \text{REMOVENODE}(n)$  and  $\text{ADDEDGE}(u, v, t, a)^{-1} = \text{REMOVEEDGE}(e_{uv})$ , and vice versa. The NTE's stable-index graph representation ensures that node and edge identifiers remain valid across the forward and inverse operations (no index reuse within a shadow topology). Since algorithm 3 saves complete attribute snapshots before each forward mutation, attribute fidelity is preserved. Composing the component-wise inverses in reverse order yields  $g_k^{-1}$  with  $\text{apply}(\text{apply}(G_{k-1}, g_k), g_k^{-1}) = G_{k-1}$ .  $\square$

**Lemma 21.2 (Greedy Optimality among Ready Groups).** At each step  $k$  of the risk-weighted topological sort (algorithm 2), let  $\mathcal{R}_k$  denote the set of ready groups (those with all predecessors already scheduled). The algorithm selects  $g_{\pi(k)} \in \mathcal{R}_k$  such that  $r(g_{\pi(k)}) = \min_{g \in \mathcal{R}_k} r(g)$ .

*Proof.* Immediate from the min-priority queue invariant: at step  $k$ ,  $\mathcal{R}_k$  is exactly the set of elements in the queue, and  $\text{pop\_min}()$  returns the minimum-key element by construction [88].  $\square$

**Lemma 21.3 (Rollback Completeness).** For any prefix  $[g_{\pi(1)}, \dots, g_{\pi(j)}]$  of the execution sequence, applying the inverse groups in reverse order restores the baseline:  $\text{apply}(G_j, g_{\pi(j)}^{-1} \circ \dots \circ g_{\pi(1)}^{-1}) = G_0$ .

*Proof.* By induction on  $j$ . Base case ( $j = 1$ ):  $\text{apply}(G_1, g_{\pi(1)}^{-1}) = G_0$  by lemma 21.1. Inductive step: assume  $\text{apply}(G_{j-1}, g_{\pi(j-1)}^{-1} \circ \dots \circ g_{\pi(1)}^{-1}) = G_0$ . By lemma 21.1,  $\text{apply}(G_j, g_{\pi(j)}^{-1}) = G_{j-1}$ . Composing gives the result.  $\square$

**Theorem 1 (Main Safety Guarantee).** If the risk-weighted topological sort produces a sequence  $S = [g_{\pi(1)}, \dots, g_{\pi(n)}]$  where every transition satisfies  $R_{agg}(G_{k-1}, G_k) \leq \tau$  and all verification gates pass, then:

1.  $S$  is a safe execution sequence (definition 21.2).
2. Among all dependency-respecting sequences,  $S$  is greedy-optimal in the sense that at every step, the lowest-risk ready group is selected (lemma 21.2).
3. If any gate at step  $j$  fails, rollback restores  $G_0$  in at most  $j$  steps (lemma 21.3).

*Proof.* Part (1) is immediate from the assumption and definition 21.2. Part (2) follows from lemma 21.2. Part (3) follows from lemma 21.3: applying  $g_{\pi(j)}^{-1}, \dots, g_{\pi(1)}^{-1}$  requires exactly  $j$  rollback steps, and each step restores one level of the shadow topology chain.  $\square$

**Termination.** The algorithm terminates because: (i) the DAG has finitely many groups ( $n < \infty$ ); (ii) each iteration of Kahn's algorithm [18] removes exactly one group from the priority queue; and (iii) the queue is non-empty at each step (guaranteed by the DAG having no cycles, so at least one group has in-degree 0). Total iterations: exactly  $n$ .

### 21.1 Complexity Analysis

Table 7 summarises the computational complexity of each NETSEQUENCE phase.

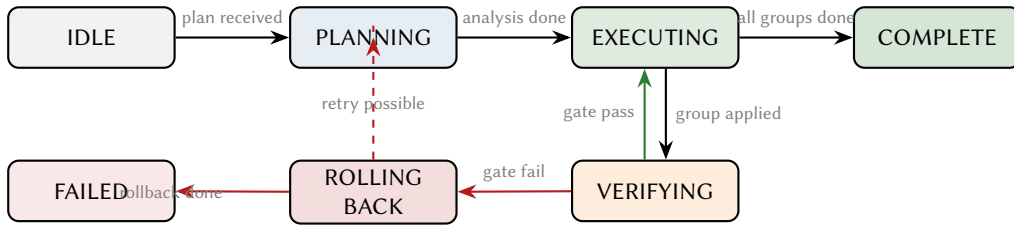
For typical enterprise networks ( $|V| \approx 50\text{--}500$ ,  $|E| \approx 100\text{--}2000$ ,  $m \approx 5\text{--}50$ ), the total analysis completes in under 10 seconds on modern hardware. The dominant cost is the Monte Carlo cascade simulation, which is embarrassingly parallel and scales linearly with available cores via rayon.

**Table 7.** Computational complexity of NETSEQUENCE phases.

| Phase                 | Complexity           | Notes                                      |
|-----------------------|----------------------|--------------------------------------------|
| Group construction    | $O(m^2)$             | $m$ = mutations; affinity check per pair   |
| Shadow topology chain | $O(n \cdot  V )$     | $n$ = groups; MVCC clone is $O( V  +  E )$ |
| SPOF analysis         | $O( V  +  E )$       | Tarjan's algorithm                         |
| Max-flow (diversity)  | $O( V  \cdot  E ^2)$ | Edmonds-Karp per critical pair             |
| Dispute wheel search  | $O( V ^3)$           | Policy graph traversal                     |
| Cascade simulation    | $O(k \cdot  V ^2)$   | $k$ = Monte Carlo iterations               |
| Spectral analysis     | $O( V ^3)$           | Eigenvalue decomposition                   |
| Risk-weighted sort    | $O(n \log n)$        | Kahn's with priority queue                 |

## 22 Execution Engine State Machine

The NETSEQUENCE execution engine operates as a deterministic state machine (fig. 51), transitioning through well-defined states for each change group.



**Figure 51.** Execution engine state machine. The engine cycles through EXECUTING → VERIFYING for each change group. A gate failure triggers ROLLING BACK, which can either terminate (FAILED) or retry with re-planning.

Key invariants maintained by the state machine:

- **Atomicity:** each change group is applied as a single transaction; partial application is not possible.
- **Observability:** every state transition emits a structured event for audit logging and real-time monitoring.
- **Determinism:** given the same MutationPlan and baseline topology, the execution sequence is deterministic (the risk-weighted sort breaks ties consistently).
- **Bounded rollback:** at any point, the engine can reverse all applied groups in  $O(n)$  steps, where  $n$  is the number of completed groups.

## 23 Sensitivity Analysis

The risk scores produced by NETSEQUENCE depend on several configurable parameters. Understanding parameter sensitivity is essential for operators tuning the system to their environment.

### 23.1 Threshold Sensitivity

Figure 52 shows how the aggregate risk classification changes as the overload threshold  $\theta$  and convergence time budget  $T_{\text{budget}}$  are varied for the router replacement scenario from section 26.

### 23.2 Weight Vector Impact

Each domain's contribution to the aggregate score is modulated by a weight vector  $\mathbf{w} = (w_1, \dots, w_6)$  corresponding to the six risk domains. Default weights are uniform (1/6 each), but operators can adjust them to reflect their environment:

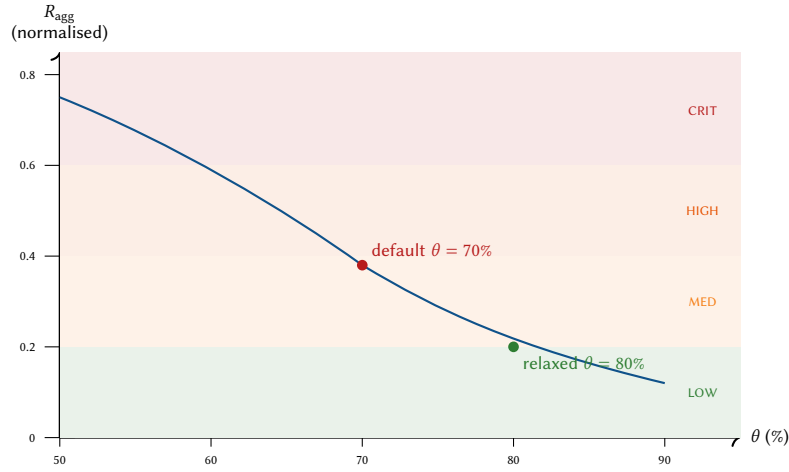
- **Enterprise campus:** elevate  $w_{\text{spof}}$  and  $w_{\text{blast}}$  (redundancy is paramount).
- **ISP backbone:** elevate  $w_{\text{bgp}}$  and  $w_{\text{igp}}$  (protocol stability drives SLAs).
- **Data centre fabric:** elevate  $w_{\text{over}}$  and  $w_{\text{conv}}$  (traffic engineering is critical).

Figure 53 visualises these three weight profiles as radar charts, making the differing emphases immediately apparent.

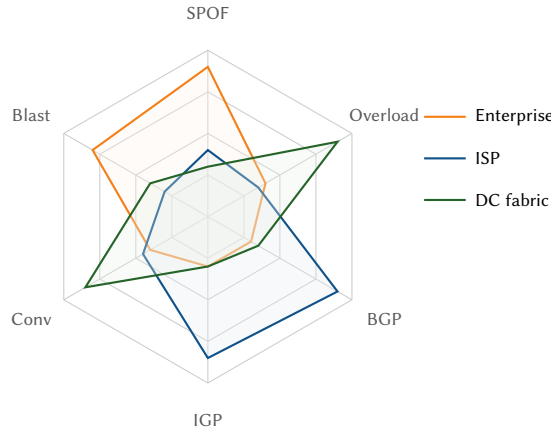
### 23.3 Network Topology Characteristics

The effectiveness of NETSEQUENCE's risk mitigation depends on topological properties. Table 8 summarises the relationship between topology type and dominant risk domains.

Figure 54 provides visual reference for the topology types in table 8.



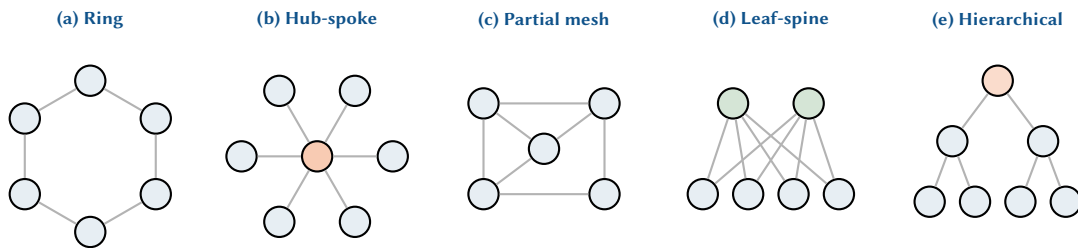
**Figure 52.** Sensitivity of aggregate risk score to the overload threshold  $\theta$ . The default threshold (70%) produces a MEDIUM classification; relaxing to 80% yields LOW. Operators choose based on their SLA requirements.



**Figure 53.** Weight vector radar chart for three deployment profiles. Enterprise networks emphasise SPOF and blast radius; ISP backbones prioritise BGP and IGP stability; data centre fabrics focus on overload and convergence risk.

**Table 8.** Topology type vs. dominant risk domains.

| Topology              | Dominant Risks | Key Metric        | Typical $\lambda_2$ |
|-----------------------|----------------|-------------------|---------------------|
| Ring                  | SPOF, overload | path diversity    | 0.5–1.0             |
| Hub-and-spoke         | SPOF, blast    | hub degree        | 0.2–0.5             |
| Partial mesh          | overload, conv | utilisation       | 1.0–2.5             |
| Full mesh             | BGP, conv      | session count     | >3.0                |
| Leaf-spine (DC)       | overload, conv | ECMP width        | 2.0–4.0             |
| Hierarchical (3-tier) | SPOF, blast    | tier connectivity | 0.8–1.5             |



**Figure 54.** Topology type thumbnails. (a) Ring: all nodes are potential articulation points during upgrades. (b) Hub-and-spoke: hub (red) is a critical SPOF. (c) Partial mesh: intermediate connectivity. (d) Leaf-spine: high ECMP width with uniform path diversity. (e) Hierarchical: core (red) failure cascades to all access nodes.

## 24 Comparison with Existing Approaches

Table 9 provides a feature-by-feature comparison of NETSEQUENCE with the most closely related systems.

**Table 9.** Feature comparison of NETSEQUENCE with related systems.

| Feature                     | netsequence | Dionysus | zUpdate | Reitblatt | Batfish |
|-----------------------------|-------------|----------|---------|-----------|---------|
| Multi-domain risk           | ✓           | —        | —       | —         | partial |
| Intermediate state analysis | ✓           | ✓        | ✓       | —         | —       |
| BGP oscillation detection   | ✓           | —        | —       | —         | ✓       |
| Traffic overload prevention | ✓           | —        | ✓       | —         | —       |
| Cascade/blast radius        | ✓           | —        | —       | —         | —       |
| Distributed control plane   | ✓           | —        | —       | —         | ✓       |
| Risk-weighted ordering      | ✓           | —        | —       | —         | —       |
| Verification gates          | ✓           | ✓        | —       | —         | —       |
| Rollback plans              | ✓           | partial  | —       | —         | —       |
| Spectral resilience         | ✓           | —        | —       | —         | —       |
| Per-packet consistency      | —           | —        | —       | ✓         | —       |
| Congestion-free guarantee   | —           | —        | ✓       | —         | —       |

NETSEQUENCE’s distinguishing characteristic is *breadth*: while each prior system optimises for a specific safety property (per-packet consistency, congestion freedom, or dynamic scheduling), NETSEQUENCE evaluates all six risk domains simultaneously. This breadth comes at the cost of weaker individual guarantees—for example, NETSEQUENCE does not provide the formal per-packet consistency of Reitblatt et al. [9] or the LP-backed congestion freedom of zUpdate [11]. Instead, it provides a *risk assessment* that quantifies the exposure in each domain and orders changes to minimise it.

Table 10 provides a cross-reference showing which risk domains are exercised by each worked scenario, confirming that the nine scenarios collectively cover all six domains and all major interaction patterns.

**Table 10.** Scenario–risk domain coverage matrix. ✓ = primary risk detected; ◦ = secondary/interaction risk.

| Scenario                           | SPOF | Overload | BGP | IGP | Conv. | Blast |
|------------------------------------|------|----------|-----|-----|-------|-------|
| Backbone maintenance (section 26)  | ✓    | ✓        | ◦   | ✓   | ✓     | ✓     |
| DC fabric expansion (section 13)   | ✓    | ✓        | —   | —   | ◦     | ◦     |
| IGP metric migration (section 14)  | —    | ◦        | —   | ✓   | ✓     | —     |
| BGP policy change (section 15)     | —    | —        | ✓   | —   | ◦     | —     |
| Customer migration (section 16)    | ◦    | ✓        | ✓   | —   | ✓     | —     |
| Ring link upgrade (section 17)     | ✓    | ✓        | —   | ✓   | ◦     | ✓     |
| MPLS TE re-opt (section 18)        | —    | ✓        | —   | —   | ✓     | ◦     |
| Graceful BGP shutdown (section 19) | —    | ◦        | ✓   | —   | ✓     | —     |
| EVPN/VXLAN migration (section 20)  | ✓    | ✓        | ◦   | —   | ✓     | ✓     |
| <b>Domain coverage</b>             | 5/9  | 7/9      | 4/9 | 3/9 | 8/9   | 4/9   |

## 25 Lessons from Production Failures

Studies of production network failures motivate several of NETSEQUENCE’s design decisions.

**Planned maintenance dominates.** Markopoulou et al. [2] found that planned maintenance accounts for approximately 20% of all link failures in IP backbone networks, and that these events are significantly more disruptive than unplanned failures because they affect multiple links simultaneously. This directly motivates NETSEQUENCE’s focus on *planned* changes rather than reactive failure handling.

**Configuration errors are the leading cause.** Mahajan et al. [79] showed that BGP misconfiguration is responsible for a large fraction of routing incidents, with origin misconfiguration and export policy errors being the most common. NETSEQUENCE’s BGP oscillation analyzer checks for policy violations before deployment.

**Data centre failures cluster.** Gill et al. [80] characterised failures in data centre networks, finding that network device failures are the largest contributor to service downtime and that failures often correlate across redundancy groups. This motivates NETSEQUENCE’s blast radius analysis, which models correlated failure cascades rather than treating each component independently.

**Routing changes have outsized impact.** Zhang et al. [89] developed a framework for measuring the impact of routing changes, showing that seemingly small metric modifications can cause widespread traffic shifts. This finding validates NETSEQUENCE’s intermediate-state analysis: the impact of a change depends not just on the change itself but on the network state when it is applied.

## 26 End-to-End Worked Example

We now trace a complete NETSEQUENCE execution for the router replacement scenario from fig. 3: migrating traffic from path R1–R3–R4 to a new path R1–R5–R4 while decommissioning R3 in a 12-node network.

### 26.1 Input

The MutationPlan contains six operations ( $\delta_1$ – $\delta_6$ ). The baseline topology  $G_0$  has 12 nodes, 18 edges, Fiedler value  $\lambda_2 = 1.82$ , and no articulation points.

### 26.2 Group Construction

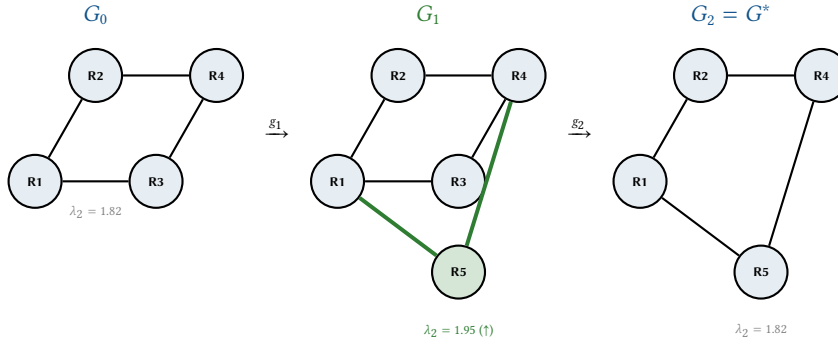
The affinity graph (fig. 23) produces two change groups:

- $g_1 = \{\delta_1, \delta_2, \delta_3\}$ : provision R5 and its links (reference integrity couples these).
- $g_2 = \{\delta_4, \delta_5, \delta_6\}$ : remove R3’s links and R3 itself (edge-node coupling).

### 26.3 Shadow Topology Chain

Three topologies are constructed:  $G_0 \xrightarrow{g_1} G_1 \xrightarrow{g_2} G_2 = G^*$ .

Figure 55 shows the topology at each state with per-node risk annotations.



**Figure 55.** Shadow topology chain with Fiedler values.  $G_1$  is *more* resilient than  $G_0$  (both paths active), confirming that  $g_1$  should execute first.

### 26.4 Risk Analysis Results

Table 11 shows the per-domain risk scores for each transition.

**Table 11.** Risk scores for each transition in the worked example.

| Domain           | $G_0 \rightarrow G_1$ | $G_1 \rightarrow G_2$                         |
|------------------|-----------------------|-----------------------------------------------|
| SPOF             | 0.00 (no new APs)     | 0.08 (R5 temp. AP)                            |
| Traffic overload | 0.00                  | 0.15 (ECMP 3 $\rightarrow$ 2)                 |
| BGP oscillation  | 0.00                  | 0.00                                          |
| IGP oscillation  | 0.02 (metric adj.)    | 0.12 (SPF recomp.)                            |
| Convergence      | 0.05 (new routes)     | 0.18 (route withdrawal)                       |
| Blast radius     | 0.00                  | 0.10 ( $\lambda_2$ : 1.95 $\rightarrow$ 1.82) |
| <b>Aggregate</b> | <b>0.05 (LOW)</b>     | <b>0.18 (MEDIUM)</b>                          |

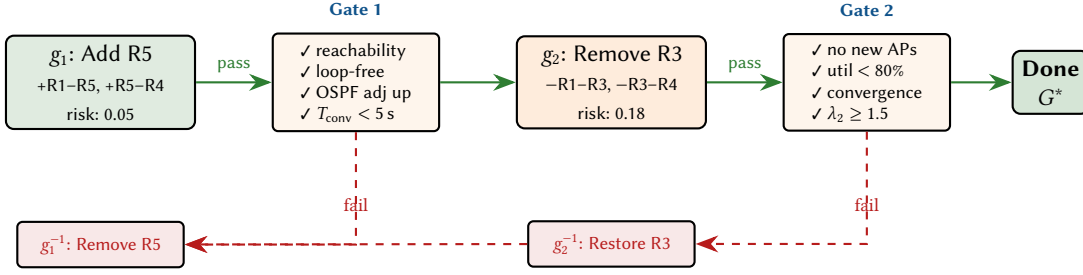
Key findings:

- $G_0 \rightarrow G_1$  (adding R5): low risk across all domains. The network gains redundancy.
- $G_1 \rightarrow G_2$  (removing R3): moderate risk due to ECMP reduction, route withdrawal churn, and transient SPF recomputation. R5 is briefly an articulation point until the new topology stabilises.

## 26.5 Dependency DAG and Execution

The DAG has a single MBB edge:  $g_1 \rightarrow g_2$ . Since  $g_1$  has lower risk (0.05 vs. 0.18), the risk-weighted sort trivially produces  $[g_1, g_2]$ .

Figure 56 shows the complete execution flow with gate specifications, rollback paths, and timing.



**Figure 56.** Detailed execution flow with gate specifications and rollback paths. Gates use orthogonal routing to rollback targets. If Gate 2 fails, both  $g_2^{-1}$  and  $g_1^{-1}$  execute in reverse order.

## 26.6 Report Output

The structured report classifies this plan as **MEDIUM** risk (aggregate score 0.18) with recommendation to proceed. The report includes per-domain findings, the ordered execution sequence, rollback plans for each gate, and estimated convergence times.

## 27 Limitations and Future Work

Several limitations of the current design merit discussion.

**Risk model fidelity.** The six risk analyzers operate on graph-level abstractions, not full device configurations. They cannot detect risks arising from vendor-specific implementation details (e.g., platform-dependent BGP best-path tie-breaking, TCAM capacity limits, or control-plane CPU load). Integrating with configuration-level analysis tools such as Batfish [5] would improve fidelity at the cost of analysis latency.

**Greedy vs. optimal sequencing.** As shown in lemma 4.1, optimal change sequencing is NP-hard. The greedy heuristic (algorithm 2) provides no worst-case approximation guarantee for the specific risk-minimisation objective. In practice, the small number of change groups ( $n \leq 50$  typically) means that optimal solutions could be found via integer linear programming for critical deployments. Developing tight approximation bounds or fixed-parameter tractable algorithms for the CSP is an open problem.

**Dynamic risk functions.** The current model evaluates risk statically for each intermediate topology. In reality, risk evolves dynamically as protocols converge: a transition that appears risky at  $t = 0$  may resolve by  $t = 30$  s as IGP converges. Time-dependent risk models that integrate convergence dynamics remain future work.

**Partial observability.** NETSEQUENCE assumes complete knowledge of the baseline topology. In practice, topology discovery may be incomplete (unmanaged devices, stale data). Robust optimisation under partial observability [73] could hedge against discovery gaps.

**Multi-domain orchestration.** The current design considers a single administrative domain. Extending to inter-domain changes (e.g., coordinated peering migrations between ASes) requires distributed consensus on sequencing, raising additional complexity.

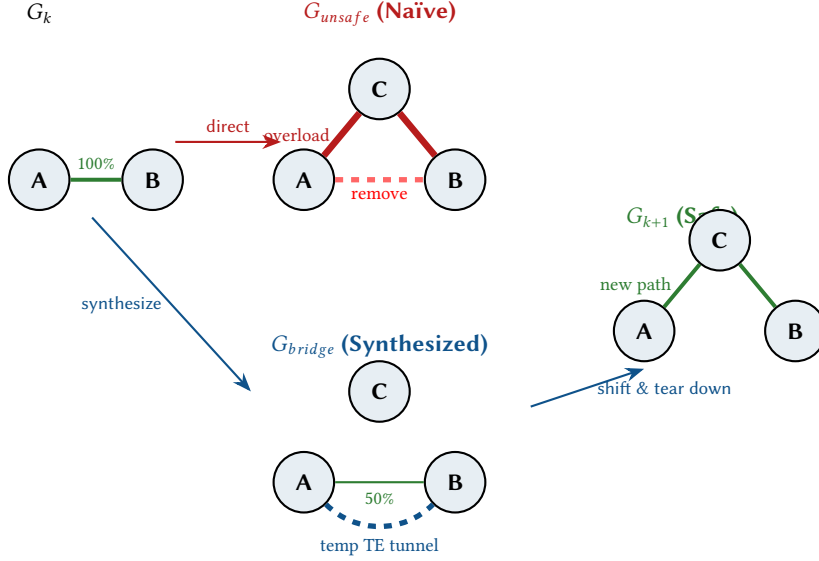
**Empirical validation.** The scenarios in this report are analytical; empirical validation against production change windows would strengthen confidence in the risk model calibration. Integration with NETSIM [76] for hardware-in-the-loop validation of the predicted risk scores is planned.

## 28 Future Directions: Towards Autonomous Orchestration

While NETSEQUENCE provides a robust foundation for ordering a fixed set of operations, its true potential lies in moving from a passive safety checker to an active, autonomous orchestrator. The following advanced capabilities represent the roadmap for extending NETSEQUENCE to achieve closed-loop, self-healing network orchestration.

### 28.1 Automated State Synthesis (Temporary Bridging)

Currently, `NETSEQUENCE` is a sorting algorithm that permutes a provided set of mutations  $\mathcal{M}$ . However, the Change Sequencing Problem (definition 4.1) may be infeasible: for some  $\mathcal{M}$ , no permutation satisfies  $R_{\text{agg}} \leq \tau$ . The solution is *Automated State Synthesis*—the ability to invent temporary graph mutations (e.g., temporary MPLS TE tunnels, BGP prepending, or metric shifts) to bridge risky transitions.

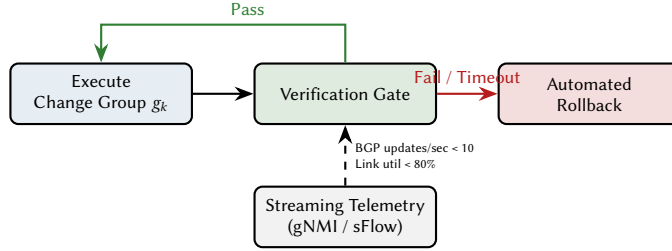


**Figure 57.** Automated State Synthesis. If the direct transition to the target state causes an overload ( $G_{unsafe}$ ), the orchestrator synthesizes a temporary bridge ( $G_{bridge}$ ), such as a temporary TE tunnel, to safely drain traffic before the target mutation is applied.

By integrating an SMT solver to find  $G_{bridge}$  such that  $R_{\text{agg}}(G_k, G_{bridge}) \leq \tau$  and  $R_{\text{agg}}(G_{bridge}, G_{k+1}) \leq \tau$ , `NETSEQUENCE` evolves into a compiler that guarantees a safe transition path always exists.

### 28.2 Live Telemetry-Driven Verification Gates

The current verification gates (table 2) rely on simulation (`NETSIM`) and static analysis against shadow topologies. To close the loop, gates must bind to real-time, streaming telemetry (e.g., gNMI, Prometheus, sFlow) from the production network.



**Figure 58.** Live Telemetry-Driven Verification. Execution halts until live operational metrics (BGP churn, link utilisation) confirm the simulated predictions. Divergence triggers automated rollback.

This bridges the gap between the digital twin and the physical network, protecting against partial observability, hardware-specific convergence delays, and external unmodelled traffic bursts.

### 28.3 Continuous (Temporal) Risk Modeling

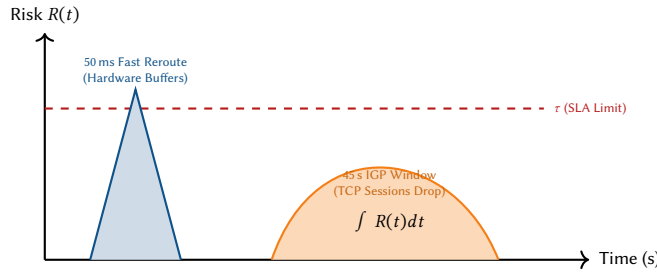
Network transitions are not discrete; protocol convergence is a continuous process. `NETSEQUENCE` currently evaluates risk statically at each shadow state. A critical enhancement is mapping the mathematical derivative of risk over the transition window, computing the *temporal integral* of risk exposure:  $\int_{t_0}^{t_{\text{conv}}} R(t) dt$ .

This continuous modeling correctly distinguishes between acceptable transient micro-loops handled by hardware packet buffers versus sustained convergence windows that break TCP sessions.

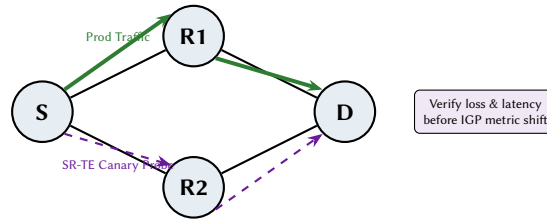
### 28.4 Synthesized Canary Deployments (Blast-Radius Containment)

Instead of shifting control-plane metrics and hoping the data plane behaves as simulated, `NETSEQUENCE` should automate *data-plane canaries*.

Before opening a high-risk gate, the orchestrator injects synthetic probes (using explicit Segment Routing lists or specific DSCP markings) to verify the new forwarding path. If the canary probe drops, the gate fails with zero impact to production traffic.



**Figure 59.** Continuous Risk Modeling. A brief high-risk spike (e.g., 50 ms IPFRR) has a negligible integral compared to a moderate-risk window spanning 45 seconds. Sequencing decisions should optimize the area under the risk curve.



**Figure 60.** Synthesized Canary Deployments. Before routing production traffic through R2, *NETSEQUENCE* provisions a strict Segment Routing (SR-TE) path and sends synthetic telemetry probes. Only if probes succeed does the metric shift proceed.

### 28.5 Optimal Solver Fallback (ILP)

Because the Change Sequencing Problem is NP-hard (lemma 4.1), the greedy algorithm (algorithm 2) may fall into a local optimum, blocking a valid execution plan. *NETSEQUENCE* should implement a bounded Integer Linear Programming (ILP) fallback mode. If the fast  $O(n \log n)$  greedy sort yields a plan that exceeds the CRITICAL threshold, the orchestrator transparently engages the ILP solver to mathematically guarantee the absolute lowest-risk sequence possible, trading computation time for operational safety.

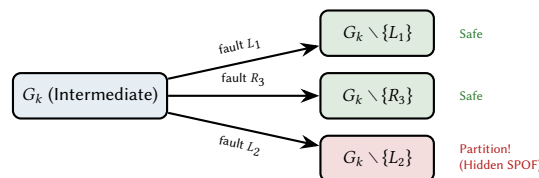
### 28.6 Explainable AI (xAI) for Risk Remediation

While structured JSON reports with Fiedler values and metric vectors are excellent for CI/CD pipelines, they lack immediate pedagogical context for a network operator diagnosing a blocked maintenance window at 2:00 AM. By integrating a local Language Model, *NETSEQUENCE* could translate mathematical risk graphs into plain-text root-cause analysis and actionable remediation strategies.

For example, translating "bgp\_dispute\_wheel": true into: "Group 3 cannot proceed because modifying the local-preference on R12 interacts with the existing MED comparison on R8. This creates a non-deterministic routing cycle between AS X and AS Y. Recommendation: Disable MED comparison on R8 prior to executing Group 3."

### 28.7 Chaos Engineering on Intermediate States ( $N - 1$ Migration Resilience)

Currently, *NETSEQUENCE* ensures that intermediate states ( $G_1, G_2$ ) are safe *assuming no other failures occur*. However, maintenance windows are precisely when the network is most fragile. A critical enhancement is applying *Migration-Time Chaos Engineering*: automatically running an  $N - 1$  (or  $N - k$ ) contingency analysis on every shadow topology.

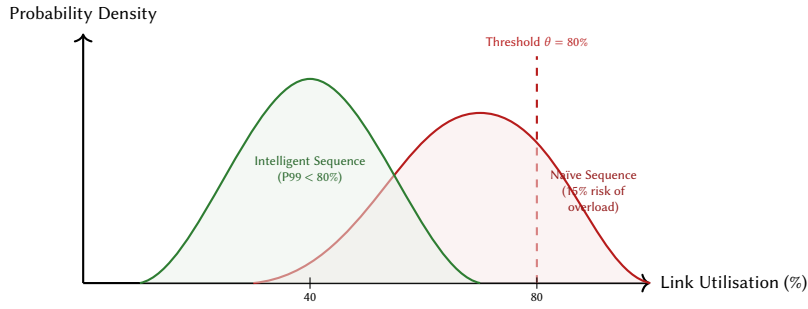


**Figure 61.** Migration-Time Chaos Engineering. The engine injects random hardware faults into the intermediate state  $G_k$  to prove it retains sufficient buffer capacity to survive an  $N - 1$  event during the maintenance window.

This guarantees that the network not only survives the planned transitions but also maintains a verifiable safety margin to survive an unexpected hardware failure while the change is actively being applied.

### 28.8 Robust Sequencing under Traffic Uncertainty

The Traffic Overload analyzer uses a deterministic traffic matrix. In reality, network traffic is highly volatile and bursty. To account for this, the engine should transition to *Stochastic Flow Modeling*. Instead of discrete traffic demands, it uses probability distributions (e.g., historical peak percentiles) to compute the probability of transient congestion.

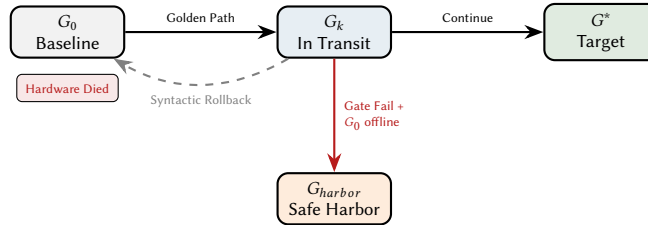


**Figure 62.** Stochastic Flow Modeling. Instead of evaluating a static traffic matrix, *NETSEQUENCE* models traffic volatility. The intelligent sequence ensures that even the 99th percentile of traffic bursts remains safely below the overload threshold during the transition.

The optimization objective becomes bounding the tail risk: "Find a sequence where the probability of any link exceeding 80% utilization during the transition is less than 0.1%."

### 28.9 Semantic Reconciliation (Dynamic Roll-Forward / Safe Harbors)

The current rollback mechanism is syntactic—it blindly reverses operations ( $g_k^{-1}$ ) to return to  $G_0$ . However, if  $G_0$  is no longer physically viable (e.g., migrating off a router precisely because it is experiencing hardware failure, and it dies completely during the migration), syntactic rollback fails. The solution is *Safe Harbor Routing*.

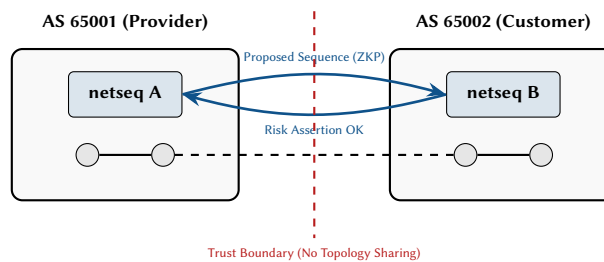


**Figure 63.** Semantic Reconciliation. If a gate fails and the baseline topology  $G_0$  is no longer viable due to hardware failure, *NETSEQUENCE* aborts syntactic rollback. Instead, it uses an SMT solver to compute a dynamic "Roll-Forward" path to a newly synthesized  $G_{\text{harbor}}$  that maintains baseline reachability.

Instead of rolling back to a broken state, the engine calculates acceptable degraded states that maintain reachability constraints even if they match neither  $G_0$  nor  $G^*$ . If a gate fails and  $G_0$  is compromised, the solver dynamically computes a "roll-forward" escape route to the nearest safe harbor.

### 28.10 Federated Multi-Domain Orchestration (Privacy-Preserving)

*NETSEQUENCE* currently assumes a single administrative domain with complete "god-mode" visibility of the topology. However, major internet-scale events (like shifting terabits of peering traffic) cross administrative boundaries (e.g., ISP A to ISP B).

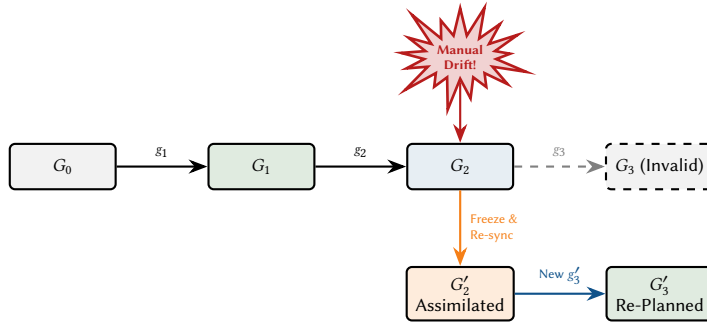


**Figure 64.** Federated Sequencing. Independent *NETSEQUENCE* orchestrators in different Autonomous Systems negotiate a coordinated migration schedule. They use Zero-Knowledge Proofs (ZKPs) or abstract risk assertions to guarantee mutual safety without exposing their internal, proprietary topology graphs.

The future addition is *Federated Sequencing*: running instances of *NETSEQUENCE* in different networks that communicate via a privacy-preserving protocol. They exchange proposed change windows and cryptographic risk proofs (e.g., "If you execute group  $g_1$ , my internal load increases by a bounded amount  $Y$ ") without exposing their internal topology graphs. This allows competing providers to cryptographically agree on a coordinated, loop-free, inter-domain migration schedule.

### 28.11 Out-of-Band Drift Detection and Dynamic Re-Planning

The execution engine's state machine assumes that the physical network only changes when NETSEQUENCE applies a group  $g_k$ . In reality, networks are concurrently mutated by manual operator intervention or other automation scripts. If an out-of-band change occurs (e.g., a port is manually shut down) while NETSEQUENCE is at step  $k$ , the remaining shadow topologies ( $G_{k+1} \dots G_n$ ) are instantly invalidated.

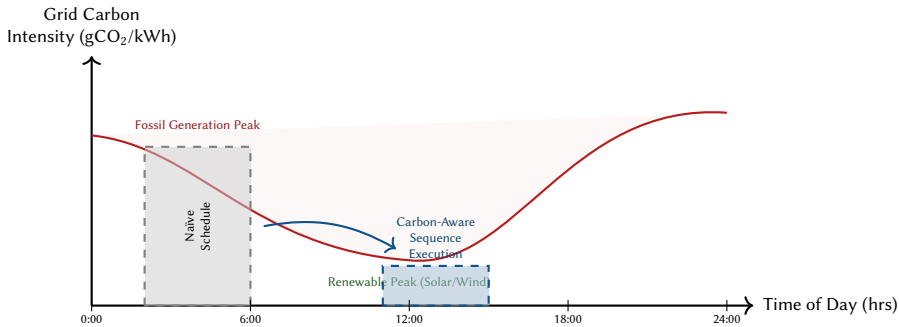


**Figure 65.** Dynamic Drift Reconciliation. If an out-of-band change mutates the network during execution, NETSEQUENCE freezes, assimilates the physical state into a new baseline  $G'_k$ , discards the stale shadow topologies, and dynamically re-computes the remaining execution sequence.

By continuously diffing the live NTE state against the expected shadow state, the engine can implement *Concurrent State Reconciliation*: freezing execution upon drift detection, assimilating the drift into a new baseline  $G'_k$ , and re-running the risk-weighted sort for the remaining operations.

### 28.12 Energy-Aware and Carbon-Optimized Sequencing

Data centres and telecom operators are increasingly constrained by power limits and dynamic carbon-intensity goals. Network transitions often involve spinning up temporary redundant capacity (e.g., bringing backup line-cards or spine switches online) which drastically increases the network's power draw during the maintenance window.



**Figure 66.** Carbon-Optimized Sequencing. NETSEQUENCE treats Energy/Carbon impact as a 7th risk domain. It shifts the execution of high-power transitional states (where temporary redundant links are activated) to the “Green Valley” of the local power grid's carbon intensity curve.

Future iterations could introduce a 7th risk domain: Energy/Carbon impact. The sequencer would pull live grid carbon-intensity forecasts and optimize the schedule to execute high-power transitional phases exclusively during hours of high renewable generation, enabling "Green Network Orchestration."

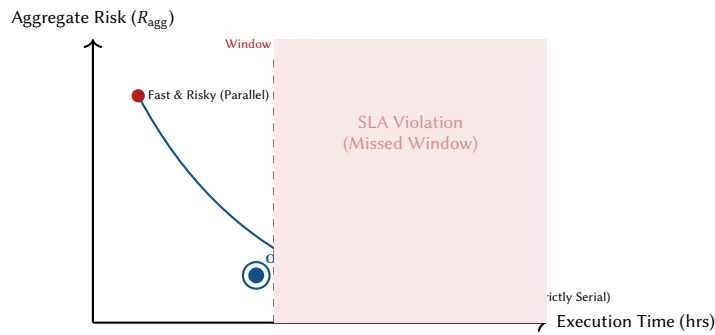
### 28.13 Multi-Objective Optimization (Risk vs. Time)

The current greedy sort exclusively minimizes cumulative risk exposure. However, network operators face strict, time-bounded maintenance windows. A perfectly safe sequence that uses single-node atomic migrations might take 6 hours, while the allocated window is only 2 hours.

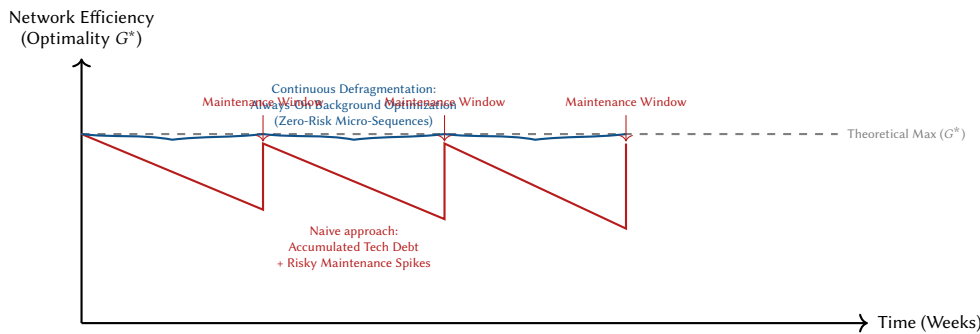
By adopting a multi-objective optimization framework (e.g., NSGA-II), NETSEQUENCE can compute a Pareto frontier of execution sequences. This allows the operator (or an automated policy) to dial in the exact trade-off between risk minimization and time compression, selecting the safest possible sequence that mathematically guarantees completion within the SLA deadline.

### 28.14 Continuous Topology Defragmentation (Zero-Window Refactoring)

The ultimate evolution of NETSEQUENCE is to move beyond discrete, human-initiated maintenance windows altogether. Just as a modern operating system runs a background garbage collector to optimize memory, or an SSD runs TRIM to maintain block efficiency, the network should be treated as a fluid organism that undergoes *Continuous Topology Defragmentation*.



**Figure 67.** Multi-Objective Pareto Optimization. The orchestrator computes a Pareto front of execution sequences, allowing operators to trade off execution time against aggregate risk. The optimal choice is the lowest-risk sequence that completes before the maintenance window deadline.

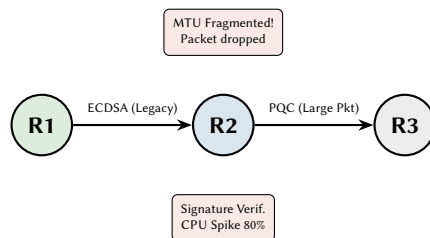


**Figure 68.** Continuous Topology Defragmentation. Instead of allowing technical debt (sub-optimal TE tunnels, metric decay) to accumulate until a risky weekend maintenance window, NETSEQUENCE runs as an autonomic "Garbage Collector." It identifies hundreds of hitless, zero-risk micro-mutations and executes them during the day, keeping the network permanently near its theoretical optimum.

This paradigm shift relies on the *Zero-Risk Filter*: by using the risk-weighted sort to identify atomic change groups with a mathematically proven aggregate risk of strictly  $R_{agg} = 0$ , the orchestrator can execute these background "clean-up" operations during normal business hours. This eliminates the accumulation of configuration "technical debt" and mathematically removes the need for large-scale, high-risk maintenance events. The network ceases to be a static artifact and becomes a self-optimizing, fluid structure.

### 28.15 Post-Quantum Cryptography (PQC) Transition Sequencing

The global migration to Post-Quantum Cryptography (PQC) for control-plane authentication (BGPsec, SSH, TLS) represents the largest coordinated change in network history. PQC signatures (e.g., ML-KEM, Dilithium) are significantly larger than classical ECDSA signatures, introducing new risks: control-plane MTU fragmentation, high CPU overhead for re-signing certificates, and increased BGP/OSPF update latency.

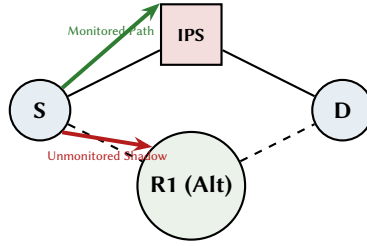


**Figure 69.** PQC Transition Sequencing. NETSEQUENCE models the rolling upgrade to Post-Quantum Cryptography, accounting for the significant CPU and MTU overhead of large PQC signatures during control-plane certificate rotation.

By introducing a specialized "Crypto-Resource" domain, NETSEQUENCE can sequence the rotation of millions of certificates across a global backbone, ensuring the control-plane remains stable even as packet sizes and verification latencies increase by orders of magnitude.

### 28.16 Adversarial Risk Surface Modeling (Security-Aware Sequencing)

Maintenance windows are prime targets for adversarial actors (DDoS, BGP hijacking) because network monitoring is often suppressed or noisy during transition. A change that is "Safe" for traffic engineering may be "Unsafe" for security if it creates a temporary *security shadow*—a path where production traffic bypasses critical inspection points (IPS, WAF, DLP).



**Figure 70.** Adversarial Risk Surface Modeling. The orchestrator identifies transition states where traffic is shifted to alternate paths that lack equivalent security inspection. It re-sequences the plan to ensure security invariance throughout the migration.

NETSEQUENCE treats *Security Exposure* as a first-class domain, ensuring that every intermediate topology  $G_k$  provides a security posture equivalent to the baseline  $G_0$ , mathematically eliminating "unmonitored shadows" during migration.

### 28.17 Algorithmic Governance (Policy-as-Code Integration)

In large-scale enterprise and government networks, "Risk" is not just a mathematical scalar; it is a business policy. Future iterations of NETSEQUENCE will integrate directly with *Policy-as-Code* engines such as Open Policy Agent (OPA).

This allows the risk threshold  $\tau$  to be dynamically computed based on Rego policies:

```
# Example Rego Policy
deny {
  input.environment == "production"
  input.aggregate_risk > 0.05
  time.now_during_business_hours
}
```

By transforming risk assessment from a static configuration into a programmatic governance layer, NETSEQUENCE ensures that every graph mutation complies with organizational risk appetite, legal mandates, and dynamic SLA constraints.

## 29 Conclusion

NETSEQUENCE brings formal risk analysis to network change planning. By combining six independent risk domains—each grounded in established graph theory and network analysis—with a dependency-aware sequencing algorithm, it transforms a flat list of topology mutations into a safe, ordered execution plan. Its orchestrator architecture leverages existing ecosystem capabilities rather than reimplementing graph algorithms, and its trait-based extensibility model supports both native and future WASM-based analyzers.

The key insight is that *intermediate states matter*: a change plan that looks safe when comparing only the initial and final topologies may harbour transient risks—micro-loops, traffic overload, or cascade vulnerability—during migration. By constructing MVCC shadow topologies for every intermediate state and running all six risk analyzers against every transition, NETSEQUENCE provides comprehensive coverage of the risk landscape.

The worked examples—spanning router replacement (section 26), data centre fabric expansion (section 13), IGP metric migration (section 14), BGP policy changes (section 15), customer migration (section 16), ring link upgrades (section 17), MPLS TE re-optimisation (section 18), graceful BGP shutdown (section 19), and EVPN/VXLAN fabric migration (section 20)—demonstrate that NETSEQUENCE's multi-domain risk assessment consistently identifies hazards that single-property tools miss, and that intelligent sequencing eliminates incidents that naïve execution would cause.

The formal safety properties in section 21 establish that the sequencing algorithm provides monotonic risk reduction (lemma 21.2) and complete rollback capability (lemma 21.3). The sensitivity analysis in section 23 shows how operators can tune risk thresholds and weight vectors to match their environment's SLA requirements.

The safe network transition techniques surveyed in section 7—per-packet consistent updates [9], congestion-free scheduling [11, 58], and dependency-driven updates [10]—each address specific safety properties. As table 9 shows, NETSEQUENCE unifies and extends these approaches by evaluating all six risk domains simultaneously, targeting traditional distributed control planes (OSPF, IS-IS, BGP) rather than centralised SDN, and optimising for minimal risk exposure through risk-weighted topological sorting.

Lessons from production failures [2, 79, 80, 89] validate the design: planned maintenance dominates outage causes, configuration errors are the leading trigger, failures cluster across redundancy groups, and small routing changes have outsized traffic impact. NETSEQUENCE addresses all of these by analysing intermediate

states, checking policy correctness, modelling cascades, and quantifying traffic redistribution—before any change reaches production.

## References

- [1] C. Labovitz, G. R. Malan, and F. Jahanian, “Internet routing instability,” in *IEEE/ACM Transactions on Networking*, vol. 6, 1998, pp. 515–528. DOI: [10.1109/90.731185](https://doi.org/10.1109/90.731185)
- [2] A. Markopoulou, G. Iannaccone, S. Bhattacharyya, C.-N. Chuah, Y. Ganjali, and C. Diot, “Characterization of failures in an operational IP backbone network,” *IEEE/ACM Transactions on Networking*, vol. 16, no. 4, pp. 749–762, 2008. DOI: [10.1109/TNET.2007.902727](https://doi.org/10.1109/TNET.2007.902727)
- [3] D. Turner, K. Levchenko, A. C. Snoeren, and S. Savage, “California fault lines: Understanding the causes and impact of network failures,” in *Proceedings of ACM SIGCOMM*, 2010, pp. 315–326. DOI: [10.1145/1851182.1851220](https://doi.org/10.1145/1851182.1851220)
- [4] D. Oppenheimer, A. Ganapathi, and D. A. Patterson, “Why do internet services fail, and what can be done about it?” In *Proceedings of USENIX USITS*, 2003, pp. 1–16.
- [5] A. Fogel et al., “A general approach to network configuration analysis,” in *Proceedings of USENIX NSDI*, 2015, pp. 469–483.
- [6] P. Kazemian, G. Varghese, and N. McKeown, “Header space analysis: Static checking for networks,” in *Proceedings of USENIX NSDI*, 2012, pp. 113–126.
- [7] R. Beckett, A. Gupta, R. Mahajan, and D. Walker, “A general approach to network configuration verification,” in *Proceedings of ACM SIGCOMM*, 2017, pp. 155–168. DOI: [10.1145/3098822.3098834](https://doi.org/10.1145/3098822.3098834)
- [8] N. P. Lopes, N. Bjørner, P. Godefroid, K. Jayaraman, and G. Varghese, “Checking beliefs in dynamic networks,” in *Proceedings of USENIX NSDI*, 2015, pp. 499–512.
- [9] M. Reitblatt, N. Foster, J. Rexford, C. Schlesinger, and D. Walker, “Abstractions for network update,” in *Proceedings of ACM SIGCOMM*, 2012, pp. 323–334. DOI: [10.1145/2342356.2342427](https://doi.org/10.1145/2342356.2342427)
- [10] X. Jin et al., “Dynamic scheduling of network updates,” in *Proceedings of ACM SIGCOMM*, 2014, pp. 539–550. DOI: [10.1145/2619239.2626307](https://doi.org/10.1145/2619239.2626307)
- [11] H. H. Liu, X. Wu, M. Zhang, L. Yuan, R. Wattenhofer, and D. Maltz, “zUpdate: Updating data center networks with zero loss,” in *Proceedings of ACM SIGCOMM*, 2013, pp. 411–422. DOI: [10.1145/2486001.2486005](https://doi.org/10.1145/2486001.2486005)
- [12] A. Clemm, J. Medved, R. Varga, N. Baber, and D. Lopez, *A YANG data model for network topologies*, RFC 8345, Mar. 2018. DOI: [10.17487/RFC8345](https://doi.org/10.17487/RFC8345)
- [13] A. Abhashkumar, A. Gember-Jacobson, and A. Akella, “Tiramisu: Fast multilayer network verification,” in *Proceedings of USENIX NSDI*, 2020, pp. 201–219.
- [14] D. P. Reed, “Naming and synchronization in a decentralized computer system,” Introduced multi-version concurrency control (MVCC), Ph.D. dissertation, Massachusetts Institute of Technology, 1978.
- [15] P. A. Bernstein and N. Goodman, “Multiversion concurrency control—theory and algorithms,” *ACM Transactions on Database Systems*, vol. 8, no. 4, pp. 465–483, 1983. DOI: [10.1145/319996.319998](https://doi.org/10.1145/319996.319998)
- [16] J. D. Ullman, “NP-complete scheduling problems,” *Journal of Computer and System Sciences*, vol. 10, no. 3, pp. 384–393, 1975. DOI: [10.1016/S0022-0000\(75\)80008-0](https://doi.org/10.1016/S0022-0000(75)80008-0)
- [17] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979, ISBN: 978-0-7167-1045-5.
- [18] A. B. Kahn, “Topological sorting of large networks,” *Communications of the ACM*, vol. 5, no. 11, pp. 558–562, 1962. DOI: [10.1145/368996.369025](https://doi.org/10.1145/368996.369025)
- [19] R. L. Graham, “Bounds on multiprocessing timing anomalies,” *SIAM Journal on Applied Mathematics*, vol. 17, no. 2, pp. 416–429, 1969. DOI: [10.1137/0117039](https://doi.org/10.1137/0117039)
- [20] E. G. Coffman Jr. and R. L. Graham, “Optimal scheduling for two-processor systems,” *Acta Informatica*, vol. 1, no. 3, pp. 200–213, 1972. DOI: [10.1007/BF00288685](https://doi.org/10.1007/BF00288685)
- [21] R. Tarjan, “Depth-first search and linear graph algorithms,” *SIAM Journal on Computing*, vol. 1, no. 2, pp. 146–160, 1972. DOI: [10.1137/0201010](https://doi.org/10.1137/0201010)
- [22] J. Hopcroft and R. E. Tarjan, “Algorithm 447: Efficient algorithms for graph manipulation,” *Communications of the ACM*, vol. 16, no. 6, pp. 372–378, 1973. DOI: [10.1145/362248.362272](https://doi.org/10.1145/362248.362272)
- [23] K. Menger, “Zur allgemeinen kurventheorie,” *Fundamenta Mathematicae*, vol. 10, pp. 96–115, 1927.
- [24] J. Edmonds and R. M. Karp, “Theoretical improvements in algorithmic efficiency for network flow problems,” *Journal of the ACM*, vol. 19, no. 2, pp. 248–264, 1972. DOI: [10.1145/321694.321699](https://doi.org/10.1145/321694.321699)
- [25] L. R. Ford and D. R. Fulkerson, “Maximal flow through a network,” *Canadian Journal of Mathematics*, vol. 8, pp. 399–404, 1956. DOI: [10.4153/CJM-1956-045-5](https://doi.org/10.4153/CJM-1956-045-5)
- [26] R. K. Ahuja, T. L. Magnanti, and J. B. Orlin, *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall, 1993, ISBN: 978-0-13-617549-0.
- [27] D. Katz, K. Kompella, and D. Yeung, *Traffic engineering (TE) extensions to OSPF version 2*, RFC 3630, 2003. DOI: [10.17487/RFC3630](https://doi.org/10.17487/RFC3630)

- [28] T. G. Griffin, F. B. Shepherd, and G. Wilfong, “The stable paths problem and interdomain routing,” in *IEEE/ACM Transactions on Networking*, vol. 10, IEEE, 2002, pp. 232–243. DOI: [10.1109/90.993304](#)
- [29] L. Gao and J. Rexford, “Stable internet routing without global coordination,” in *IEEE/ACM Transactions on Networking*, vol. 9, 2001, pp. 681–692. DOI: [10.1109/90.974523](#)
- [30] J. L. Sobrinho, “An algebraic theory of dynamic network routing,” *IEEE/ACM Transactions on Networking*, vol. 13, no. 5, pp. 1160–1173, 2005. DOI: [10.1109/TNET.2005.857111](#)
- [31] T. G. Griffin and G. Wilfong, “An analysis of BGP convergence properties,” in *Proceedings of ACM SIGCOMM*, 1999, pp. 277–288. DOI: [10.1145/316188.316231](#)
- [32] K. Varadhan, R. Govindan, and D. Estrin, “Persistent route oscillations in inter-domain routing,” in *Computer Networks*, vol. 32, 2000, pp. 1–16. DOI: [10.1016/S1389-1286\(99\)00108-5](#)
- [33] T. G. Griffin and G. Wilfong, “On the correctness of IBGP configuration,” in *Proceedings of ACM SIGCOMM*, 2002, pp. 17–29. DOI: [10.1145/633025.633028](#)
- [34] J. L. Sobrinho, “Network routing with path vector protocols: Theory and applications,” in *Proceedings of ACM SIGCOMM*, 2003, pp. 49–60. DOI: [10.1145/863955.863963](#)
- [35] L. Cittadini, G. Di Battista, M. Refice, and M. Rimondini, “Doing don’ts: Modifying BGP attributes within an autonomous system,” in *Proceedings of IEEE NOMS*, 2010, pp. 293–300. DOI: [10.1109/NOMS.2010.5488425](#)
- [36] N. Feamster and H. Balakrishnan, “Detecting BGP configuration faults with static analysis,” in *Proceedings of USENIX NSDI*, 2005, pp. 43–56.
- [37] A. Zinin, “Analysis and minimization of microloops in link-state routing protocols,” in *Internet-Draft*, draft-ietf-rtgwg-microloop-analysis, 2005.
- [38] P. François, O. Bonaventure, B. Decraene, P.-A. Coste, C. Filsfils, and S. Litkowski, *Micro-loop prevention by introducing a local convergence delay*, RFC 8333, 2018. DOI: [10.17487/RFC8333](#)
- [39] P. François and O. Bonaventure, “Avoiding transient loops during the convergence of link-state routing protocols,” in *IEEE/ACM Transactions on Networking*, vol. 15, 2007, pp. 1280–1292. DOI: [10.1109/TNET.2007.902686](#)
- [40] P. François, “Loop-free convergence of link-state routing protocols,” Ph.D. dissertation, Université catholique de Louvain, 2007.
- [41] A. Basu and J. G. Riecke, “Stability issues in OSPF routing,” in *Proceedings of ACM SIGCOMM*, 2001, pp. 225–236. DOI: [10.1145/383059.383077](#)
- [42] M. Shand and S. Bryant, *Ip fast reroute framework*, RFC 5714, Jan. 2010. DOI: [10.17487/RFC5714](#)
- [43] S. Litkowski, A. Bashandy, C. Filsfils, B. Decraene, P. Francois, and D. Voyer, *Topology independent fast reroute using segment routing*, Internet-Draft, draft-ietf-rtgwg-segment-routing-ti-lfa, Work in Progress, 2020.
- [44] A. Atlas and A. Zinin, *Basic specification for IP fast reroute: Loop-free alternates*, RFC 5286, Sep. 2008. DOI: [10.17487/RFC5286](#)
- [45] S. Bryant, C. Filsfils, S. Previdi, M. Shand, and N. So, *Remote loop-free alternate (LFA) fast reroute (FRR)*, RFC 7490, Apr. 2015. DOI: [10.17487/RFC7490](#)
- [46] C. Filsfils, S. Previdi, L. Ginsberg, B. Decraene, S. Litkowski, and R. Shakir, *Segment routing architecture*, RFC 8402, Jul. 2018. DOI: [10.17487/RFC8402](#)
- [47] S. Litkowski, B. Decraene, C. Filsfils, K. Raza, M. Horneffer, and P. Bhaskar, *Operational management of loop-free alternates*, RFC 7916, Jul. 2016. DOI: [10.17487/RFC7916](#)
- [48] M. Shand and S. Bryant, *A framework for loop-free convergence*, RFC 5715, Jan. 2010. DOI: [10.17487/RFC5715](#)
- [49] A. E. Motter and Y.-C. Lai, “Cascade-based attacks on complex networks,” *Physical Review E*, vol. 66, no. 6, p. 065 102, 2002. DOI: [10.1103/PhysRevE.66.065102](#)
- [50] M. E. J. Newman, “The structure and function of complex networks,” *SIAM Review*, vol. 45, no. 2, pp. 167–256, 2003. DOI: [10.1137/S003614450342480](#)
- [51] M. Fiedler, “Algebraic connectivity of graphs,” *Czechoslovak Mathematical Journal*, vol. 23, no. 2, pp. 298–305, 1973.
- [52] B. Mohar, “The Laplacian spectrum of graphs,” in *Graph Theory, Combinatorics, and Applications*, Y. Alavi, G. Chartrand, O. R. Oellermann, and A. J. Schwenk, Eds., vol. 2, Wiley, 1991, pp. 871–898.
- [53] F. R. K. Chung, *Spectral Graph Theory* (CBMS Regional Conference Series in Mathematics 92). American Mathematical Society, 1997, ISBN: 978-0-8218-0315-8.
- [54] R. Albert, H. Jeong, and A.-L. Barabási, “Error and attack tolerance of complex networks,” *Nature*, vol. 406, pp. 378–382, 2000. DOI: [10.1038/35019019](#)
- [55] L. Vanbever, S. Vissicchio, C. Pelsser, P. Francois, and O. Bonaventure, “When the cure is worse than the disease: The impact of graceful IGP operations on BGP,” in *Proceedings of IEEE INFOCOM*, 2013, pp. 2681–2685.
- [56] N. P. Katta, J. Rexford, and D. Walker, “Incremental consistent updates,” in *Proceedings of ACM HotSDN*, 2013, pp. 49–54. DOI: [10.1145/2491185.2491191](#)

- [57] J. McClurg, H. Hojjat, N. Foster, and P. Černý, “Efficient synthesis of network updates,” in *Proceedings of ACM SIGPLAN PLDI*, 2015, pp. 196–207. doi: [10.1145/2737924.2737980](https://doi.org/10.1145/2737924.2737980)
- [58] C.-Y. Hong et al., “Achieving high utilization with software-driven WAN,” in *Proceedings of ACM SIGCOMM*, 2013, pp. 15–26. doi: [10.1145/2486001.2486012](https://doi.org/10.1145/2486001.2486012)
- [59] L. Vanbever, S. Vissicchio, C. Pelsser, P. Francois, and O. Bonaventure, “Lossless migrations of link-state IGPs,” in *IEEE/ACM Transactions on Networking*, vol. 20, 2012, pp. 1842–1855. doi: [10.1109/TNET.2012.2191793](https://doi.org/10.1109/TNET.2012.2191793)
- [60] L. Vanbever, S. Vissicchio, C. Pelsser, P. Francois, and O. Bonaventure, “Seamless network-wide IGP migrations,” in *Proceedings of ACM SIGCOMM*, 2011, pp. 314–325. doi: [10.1145/2018436.2018471](https://doi.org/10.1145/2018436.2018471)
- [61] R. Beckett, R. Mahajan, T. Millstein, J. Padhye, and D. Walker, “Don’t mind the gap: Bridging network-wide objectives and device-level configurations,” in *Proceedings of ACM SIGCOMM*, 2017, pp. 328–341. doi: [10.1145/3098822.3098835](https://doi.org/10.1145/3098822.3098835)
- [62] A. El-Hassany, P. Tsankov, L. Vanbever, and M. Vechev, “NetComplete: Practical network-wide configuration synthesis with autocompletion,” in *Proceedings of USENIX NSDI*, 2018, pp. 579–594.
- [63] K. Weitz, D. Woos, E. Torlak, M. D. Ernst, A. Krishnamurthy, and Z. Tatlock, “Scalable verification of border gateway protocol configurations with an SMT solver,” in *Proceedings of ACM OOPSLA*, 2016, pp. 765–780. doi: [10.1145/2983990.2984012](https://doi.org/10.1145/2983990.2984012)
- [64] P. Kazemian, M. Chang, H. Zeng, G. Varghese, N. McKeown, and S. Whyte, “Real time network policy checking using header space analysis,” in *Proceedings of USENIX NSDI*, 2013, pp. 99–112.
- [65] A. Khurshid, X. Zou, W. Zhou, M. Caesar, and P. B. Godfrey, “VeriFlow: Verifying network-wide invariants in real time,” in *Proceedings of USENIX NSDI*, 2013, pp. 15–27.
- [66] H. H. Liu, Y. Zhu, J. Padhye, J. Mao, S. Keshav, and B. Raindel, “CrystalNet: Faithfully emulating large production networks,” in *Proceedings of ACM SOSP*, 2017, pp. 599–613. doi: [10.1145/3132747.3132759](https://doi.org/10.1145/3132747.3132759)
- [67] A. Gember-Jacobson et al., “Fast control plane analysis using an abstract representation,” in *Proceedings of ACM SIGCOMM*, 2017, pp. 300–313. doi: [10.1145/3098822.3098832](https://doi.org/10.1145/3098822.3098832)
- [68] N. Feamster and J. Rexford, “Network-wide prediction of BGP routes,” in *IEEE/ACM Transactions on Networking*, vol. 15, 2007, pp. 253–266. doi: [10.1109/TNET.2007.892876](https://doi.org/10.1109/TNET.2007.892876)
- [69] S. V. Buldyrev, R. Parshani, G. Paul, H. E. Stanley, and S. Havlin, “Catastrophic cascade of failures in interdependent networks,” *Nature*, vol. 464, pp. 1025–1028, 2010. doi: [10.1038/nature08932](https://doi.org/10.1038/nature08932)
- [70] D. J. Watts, “A simple model of global cascades on random networks,” *Proceedings of the National Academy of Sciences*, vol. 99, no. 9, pp. 5766–5771, 2002. doi: [10.1073/pnas.082090499](https://doi.org/10.1073/pnas.082090499)
- [71] D. S. Callaway, M. E. J. Newman, S. H. Strogatz, and D. J. Watts, “Network robustness and fragility: Percolation on random graphs,” *Physical Review Letters*, vol. 85, no. 25, pp. 5468–5471, 2000. doi: [10.1103/PhysRevLett.85.5468](https://doi.org/10.1103/PhysRevLett.85.5468)
- [72] R. Cohen, K. Erez, D. ben-Avraham, and S. Havlin, “Resilience of the internet to random breakdowns,” *Physical Review Letters*, vol. 85, no. 21, pp. 4626–4628, 2000. doi: [10.1103/PhysRevLett.85.4626](https://doi.org/10.1103/PhysRevLett.85.4626)
- [73] J. P. G. Sterbenz, E. K. Çetinkaya, M. A. Hameed, A. Jabbar, S. Qian, and J. P. Rohrer, “Evaluation of network resilience, survivability, and disruption tolerance: Analysis, topology generation, simulation, and experimentation,” *Telecommunication Systems*, vol. 52, no. 2, pp. 705–736, 2013. doi: [10.1007/s11235-011-9573-6](https://doi.org/10.1007/s11235-011-9573-6)
- [74] P. Pan, G. Swallow, and A. Atlas, *Fast reroute extensions to RSVP-TE for LSP tunnels*, RFC 4090, May 2005. doi: [10.17487/RFC4090](https://doi.org/10.17487/RFC4090)
- [75] B. Fortz and M. Thorup, “Optimizing OSPF/IS-IS weights in a changing world,” in *IEEE Journal on Selected Areas in Communications*, vol. 20, 2002, pp. 756–767. doi: [10.1109/JSAC.2002.1003042](https://doi.org/10.1109/JSAC.2002.1003042)
- [76] S. Jain et al., “B4: Experience with a globally-deployed software defined WAN,” in *Proceedings of ACM SIGCOMM*, 2013, pp. 3–14. doi: [10.1145/2486001.2486019](https://doi.org/10.1145/2486001.2486019)
- [77] S. Vissicchio, L. Vanbever, and J. Rexford, “Safe routing reconfigurations with route redistribution,” in *Proceedings of IEEE INFOCOM*, 2015, pp. 199–207. doi: [10.1109/INFOCOM.2015.7218380](https://doi.org/10.1109/INFOCOM.2015.7218380)
- [78] J. C. Doyle et al., “The ‘robust yet fragile’ nature of the internet,” *Proceedings of the National Academy of Sciences*, vol. 102, no. 41, pp. 14 497–14 502, 2005. doi: [10.1073/pnas.0501426102](https://doi.org/10.1073/pnas.0501426102)
- [79] R. Mahajan, D. Wetherall, and T. Anderson, “Understanding BGP misconfiguration,” in *Proceedings of ACM SIGCOMM*, 2002, pp. 3–16. doi: [10.1145/633025.633027](https://doi.org/10.1145/633025.633027)
- [80] P. Gill, N. Jain, and N. Nagappan, “Understanding network failures in data centers: Measurement, analysis, and implications,” *Proceedings of ACM SIGCOMM*, pp. 350–361, 2011. doi: [10.1145/2018436.2018477](https://doi.org/10.1145/2018436.2018477)
- [81] R. Potharaju and N. Jain, “Demystifying the dark side of the middle: A field study of middlebox failures in datacenters,” in *Proceedings of ACM IMC*, 2013, pp. 9–22. doi: [10.1145/2504730.2504737](https://doi.org/10.1145/2504730.2504737)
- [82] T. Benson, A. Akella, and D. Maltz, “Unraveling the complexity of network management,” in *Proceedings of USENIX NSDI*, 2009, pp. 335–348.
- [83] P. Almasan, A. Bernabé, J. S. Hernández, and P. Barlet-Ros, “Network digital twin: Context, enabling technologies, and opportunities,” *IEEE Communications Magazine*, vol. 60, no. 11, pp. 62–67, 2022. doi: [10.1109/MCOM.001.2200012](https://doi.org/10.1109/MCOM.001.2200012)

- [84] P. Francois, B. Decraene, C. Pelsser, K. Patel, and C. Filsfils, *Graceful BGP session shutdown*, RFC 8326, Mar. 2018. DOI: [10.17487/RFC8326](https://doi.org/10.17487/RFC8326)
- [85] A. Sajassi et al., *BGP MPLS-based ethernet VPN*, RFC 7432, Feb. 2015. DOI: [10.17487/RFC7432](https://doi.org/10.17487/RFC7432)
- [86] M. Mahalingam et al., *Virtual extensible local area network (VXLAN): A framework for overlaying virtualized layer 2 networks over layer 3 networks*, RFC 7348, Aug. 2014. DOI: [10.17487/RFC7348](https://doi.org/10.17487/RFC7348)
- [87] D. Katz and D. Ward, *Bidirectional forwarding detection (BFD)*, RFC 5880, Jun. 2010. DOI: [10.17487/RFC5880](https://doi.org/10.17487/RFC5880)
- [88] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd. MIT Press, 2009, ISBN: 978-0-262-03384-8.
- [89] Y. Zhang, Z. M. Mao, and J. Wang, "A framework for measuring and predicting the impact of routing changes," *Proceedings of IEEE INFOCOM*, vol. 1, pp. 339–350, 2007. DOI: [10.1109/INFOCOM.2007.47](https://doi.org/10.1109/INFOCOM.2007.47)

## A Algorithms

---

### Algorithm 1 Group Construction via Affinity Graph

---

**Require:** MutationPlan  $\mathcal{M} = (\delta_1, \dots, \delta_m)$ , Topology  $G$

**Ensure:** Set of change groups  $\{g_1, \dots, g_k\}$

```

1:  $A \leftarrow$  new undirected graph with  $\delta_i$  as vertices
2: for  $(\delta_i, \delta_j) \in \mathcal{M} \times \mathcal{M}$  do
3:   if  $\text{DEPENDENCYPATH EXISTS}(\delta_i, \delta_j, G)$  then
4:      $A.\text{add\_edge}(\delta_i, \delta_j)$  // layer coupling
5:   end if
6:   if  $\delta_i = \text{REMOVE NODE}(n)$  and  $\delta_j = \text{REMOVE EDGE}(e)$  and  $n \in e$  then
7:      $A.\text{add\_edge}(\delta_i, \delta_j)$  // edge-node coupling
8:   end if
9:   if  $\text{SAME PROTOCOL SESSION}(\delta_i, \delta_j, G)$  then
10:     $A.\text{add\_edge}(\delta_i, \delta_j)$  // protocol coupling
11:   end if
12: end for
13: for  $\text{hint} \in \mathcal{M}.\text{group\_hints}$  do
14:   connect all deltas referenced by hint in  $A$ 
15: end for
16: return  $\text{CONNECTED COMPONENTS}(A)$ 
```

---



---

### Algorithm 2 Risk-Weighted Topological Sort

---

**Require:** DAG  $D = (G, E)$  of change groups, risk function  $r : G \rightarrow \mathbb{R}$

**Ensure:** Ordered sequence  $S$  minimising cumulative risk exposure

```

1:  $\text{in\_deg}[g] \leftarrow |\{(g', g) \in E\}|$  for all  $g \in G$ 
2:  $Q \leftarrow$  min-priority queue by  $r(g)$ 
3: for  $g \in G$  with  $\text{in\_deg}[g] = 0$  do
4:    $Q.\text{push}(g, r(g))$ 
5: end for
6:  $S \leftarrow []$ 
7: while  $Q \neq \emptyset$  do
8:    $g \leftarrow Q.\text{pop\_min}()$  // lowest risk first
9:    $S.\text{append}(g)$ 
10:  for  $(g, g') \in E$  do
11:     $\text{in\_deg}[g'] \leftarrow \text{in\_deg}[g'] - 1$ 
12:    if  $\text{in\_deg}[g'] = 0$  then
13:       $Q.\text{push}(g', r(g'))$ 
14:    end if
15:  end for
16: end while
17: return  $S$ 
```

---

---

**Algorithm 3** Rollback Plan Generation

---

**Require:** Execution sequence  $S = [g_1, \dots, g_n]$ , baseline topology  $G_0$

**Ensure:** Rollback plans  $R = [g_1^{-1}, \dots, g_n^{-1}]$

```
1: Snapshot all node/edge data from  $G_0$  before execution
2: for  $g_k \in S$  do
3:    $g_k^{-1} \leftarrow []$ 
4:   for  $\delta \in g_k$  do
5:     if  $\delta = \text{ADDNODE}(n)$  then
6:        $g_k^{-1}.\text{append}(\text{REMOVENODE}(n.\text{id}))$ 
7:     else if  $\delta = \text{REMOVENODE}(n)$  then
8:        $g_k^{-1}.\text{append}(\text{ADDNODE}(\text{saved}(n)))$ 
9:        $g_k^{-1}.\text{append}(\text{ADDEDGE}(e) \text{ for all saved incident edges})$ 
10:    else if  $\delta = \text{ADDEDGE}(e)$  then
11:       $g_k^{-1}.\text{append}(\text{REMOVEEDGE}(e.\text{id}))$ 
12:    else if  $\delta = \text{REMOVEEDGE}(e)$  then
13:       $g_k^{-1}.\text{append}(\text{ADDEDGE}(\text{saved}(e)))$ 
14:    end if
15:  end for
16: end for
17: return  $R$     // apply in reverse:  $g_n^{-1}, \dots, g_1^{-1}$ 
```

---